

IIE6101 스케줄링이론및응용

$r_j = 0$

Chapter 4. 단일기계 스케줄링 - 작업도착시간과 준비시간이 없는 경우

+ 최적화문제 → solver

본 강의자료는 연세대학교 산업공학과 김창욱 교수의 “최적 스케줄링 공학 (Scheduling)” 교재를 바탕으로 제작되었습니다.

권순걸 (sg.kwon@yonsei.ac.kr)

시스템 모델링 및 최적화 연구실, 연세대학교 산업공학과

SYstems **M**odeling and **P**rogramming Lab
Department of Industrial Engineering, Yonsei University - **SYMPLY**
(<https://symply.yonsei.ac.kr/>)

“이 자료는 연세대학교 지능형데이터·최적화학과 IDO6001 스케줄링이론및응용 교과목 강의를 위해 제작된 것으로, 다른 용도로 사용하거나 타인과 공유할 수 없으며, 위반할 경우 그 법적 책임은 행위자 본인에게 있습니다.”

Table of contents

1. 개요와 가정

2. 건설적 알고리즘

3. 암묵적 열거법

확장 알고리즘.

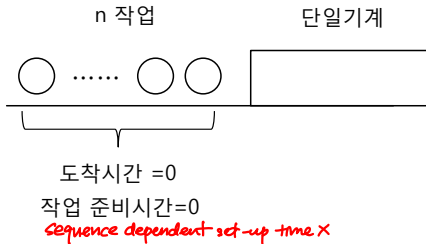
4. 메타 휴리스틱 알고리즘 → 단일하게 스케줄링

동적계획법, 분지한계법

개요와 가정 (1/4)

단일기계 스케줄링

- 다음 그림과 같이 고려해야 할 작업이 n 개 있고 모든 작업의 도착시간이 모두 0이며 ($r_i = 0$) 작업 준비시간이 존재하지 않는 조건($s_{ij} = 0$)하에서의 단일기계 일정계획을 위한 최적화 기법을 다룬다.



$$C_{max} = \text{상수} = \sum_{j=1}^n p_j$$

전처리시간 } 준비시간
 } 순차처리시간

Figure 1: 최적화를 위한 단일기계 일정계획의 조건

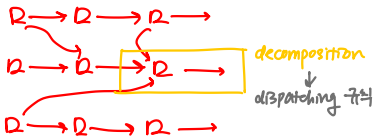
개요와 가정 (2/4)

단일기계 일정계획이 중요한 이유

- 단일 기계 일정계획이라고 하더라도 고려해야할 스케줄은 많고(예를들어 n 개 작업이 있을 때 가능한 스케줄의 수는 $n!$),이중 최적의 성능지표 값을 갖는 스케줄을 찾는것은 어려운 문제이다.
- 단일 기계 일정계획 알고리즘은 병렬 기계, 흐름 작업장, 개별(개방)작업장과 같은 복잡한 환경에서의 일정계획 문제를 풀기 위한 이론적 배경과 수단을 제공한다.

참고 cf. chapter 5

복잡한 환경의 일정계획 문제에 대해서 최적 해를 제공하는 빠른 알고리즘은 지금까지 알려져 있지 않으며, 일반적으로 문제를 단일기계 문제로 분해 (Decomposition)하여 푸는 휴리스틱 접근방법을 많이 사용한다.



개요와 가정 (3/4)

단일기계 일정계획의 가정사항

- $r_i = 0$ 이고 $s_{ij} = 0$ 이기 때문에 Makespan $C_{\max}$ 가 불변이다.
- 작업이 n 개로 고정되어 있다는 것은 더 이상 새로운 작업이 도착하지 않는다는 것을 의미하며 이는 곧 n 개의 작업을 스케줄 하여 가공하는 동안 새롭게 도착한 작업에 의해 스케줄이 변경되지 않는다는 정적인 전제조건이기도 하다. n 개의 작업스케줄. offline.
- 만일 새롭게 도착하는 작업을 계속 고려하여 스케줄을 변경하면 최적 스케줄이라는 것이 의미가 없게 되며 이런 상황(즉 작업의 도착시간이 0 이 아닌 경우)에 직면하여 일정계획을 수립하는 것을 온라인(Online) 또는 적응형(Adaptive) 스케줄링이라 부른다.
- 작업의 도착시간이 모두 0이라면 고의적으로 유휴시간(Idle time)을 발생시키는 것은 목적함수에 도움이 되지 않을 것이며, 따라서 유휴시간을 넣지 않는 스케줄만을 고려하면 된다.

개요와 가정 (4/4)

단일기계 일정계획의 성능지표와 최적화 알고리즘은 다음 표와 같다.

가정	성능지표	$\alpha \beta \gamma$ <i>change</i>	알고리즘	알고리즘 분류
$r_i = 0$ $s_{ij} = 0$	Total completion time	$1 C(F \text{ or } L)$	SPT 규칙	건설적 알고리즘 (최적화 알고리즘)
	Weighted mean flow time $((\sum_{i=1}^n \alpha_i F_i)/n)$	$1 \sum \frac{\alpha_i F_i}{n}$	WSPT 규칙	
	Maximum lateness	$1 L_{\max} \text{ or } T_{\max}$	EDD 규칙	
	Number of tardy jobs	$1 n_T$	Moore 알고리즘	
	Maximum penalty $(\max_{i=1}^n \{f_i(C_i)\})$	$1 \max_{i=1}^n \{f_i(C_i)\}$	Lawler 알고리즘	암묵적 열거법 (최적화 알고리즘)
	Any	$1 Any$	동적계획법	
	Total tardiness	$1 \sum_{i=1}^n T_i$	문제에 특화된 분지 한계법	
	Performance measure of total or average types	$1 \text{Total of Average}$	LP 완화 기반 분지 한계법	
	Any	$1 Any$	타부탐색 시뮬레이티드 어닐링 유전자 알고리즘	메타 휴리스틱

건설적 알고리즘

건설적 알고리즘은 **Polynomial** 시간 내에 최적 스케줄을 찾아낸다는 점에서 매우 효율적인 알고리즘이다. ㉠

- 따라서 건설적 알고리즘으로 풀 수 있는 일정계획 문제는 계산 복잡도 측면에서 P집합에 속한다. *
- 단일기계 스케줄링을 위한 다음의 5가지 건설적 알고리즘을 살펴보도록 한다.
 - Shortest processing time (SPT) 규칙
 - Weighted shortest processing time (WSPT) 규칙
 - Earliest due date (EDD) 규칙
 - Moore 알고리즘
 - Lawler 알고리즘

* **P**: Deterministic 튜링 기계를 이용하여 polynomial 시간 내에 풀 수 있는 알고리즘이 발견된 모든 문제 집합이다. 여기서 P는 polynomial의 약자이다.

Shortest Processing Time (SPT) 규칙 (1/4)

SPT 규칙은 단일기계 환경에서 기계의 평균 체류시간(mean flow time: $\bar{F}$)을 최소화시키기 위한 스케줄링 기법 (for 1 || $\bar{F}$) $F = W + P$ (P 는 고정, W 는 줄임수). $C = r + F$ (r 고정)
 $L = C - d$ (d 고정)

- 작업들을 가공시간이 오름차순으로 배열하여 스케줄을 생성

$$p_{i(1)} \leq p_{i(2)} \leq \dots \leq p_{i(n)} \quad : \text{SPT}$$

여기서 $p_{i(k)}$ 는 k 번째 스케줄된 작업 J_i 의 가공시간을 의미

- 2장에서 논의한 Equivalence 관계에서 알수 있듯이 SPT규칙은 1 || C , 1 || L 문제 그리고 1 || $\bar{F}$, 1 || L 문제도 최적화 한다.

Shortest Processing Time (SPT) 규칙 (2/4)

정리 4.1

n 개의 작업이 있는 단일기계 환경에서 작업의 총 체류시간(F)을 최소화하면 (또는 하려면) 다음 조건을 만족한다.

$$p_{i(1)} \leq p_{i(2)} \leq \cdots \leq p_{i(n)} \quad \leftarrow \text{SPT}$$

(증명)

- 정의에 의해서 $F = \sum_{i=1}^n F_i = \sum_{i=1}^n (W_i + p_i)$ 이며 이를 스케줄상 작업의 순거로 재정의 하면 다음과 같다.

$$F = \sum_{k=1}^n (W_{i(k)} + P_{i(k)}) = \sum_{k=1}^n W_{i(k)} + \sum_{k=1}^n p_{i(k)} \quad (1)$$

작업배치 시각

↑

상수.

- 식 (1)에서 $W_{i(k)}$ 와 $P_{i(k)}$ 는 각각 k 번째에 가공되는 작업의 가공시간과 대기시간을 뜻하며, $\sum_{k=1}^n p_{i(k)}$ 경우 가공시간의 합으로 순서와 상관없는 상수이다. 따라서 F 를 줄이려면 $\sum_{k=1}^n W_{i(k)}$ 를 줄여야 한다.

	$k=1$	$k=2$	$k=3$	$k=4$	$\dots$	$k=n$
Wick)	0	$P_{i(1)}$	$P_{i(1)}$	$P_{i(1)}$	$\vdots$	$P_{i(n)}$
			$P_{i(2)}$	$P_{i(2)}$		$P_{i(2)}$
				$P_{i(3)}$		$\vdots$
						$P_{i(n-1)}$

sum을 계산하기 위하여

$P_{i(n)}$ 추가

Shortest Processing Time (SPT) 규칙 (3/4)

정리 4.1 (continued)

- $W_{i(k)}$ 는 다음과 같이 정의할 수 있다.

$$\begin{aligned}
 W_{i(1)} &= 0 \\
 W_{i(2)} &= p_{i(1)} \\
 W_{i(3)} &= p_{i(1)} + p_{i(2)} \\
 &\vdots \\
 W_{i(k)} &= p_{i(1)} + p_{i(2)} + \cdots + p_{i(k-1)}
 \end{aligned} \tag{2}$$

- 식 (2)로부터 평균 체류시간 $\bar{F}$ 를 다음과 같이 정의할 수 있다.

$$\bar{F} = \frac{1}{n} \sum_{k=1}^n (W_{i(k)} + p_{i(k)}) = \frac{1}{n} (np_{i(1)} + (n-1)p_{i(2)} + \cdots + p_{i(n)}) = \frac{1}{n} \sum_{k=1}^n (n-k+1)p_{i(k)} \tag{3}$$

- 식 (3)로부터 우리는 가공시간이 작은 작업의 순서대로 스케줄하면 평균 체류시간이 최소화됨을 알 수 있다. Q.E.D.

Shortest Processing Time (SPT) 규칙 (4/4)

예제 4.1

다음의 (1 || F) 문제를 풀어보자.

작업	1	2	3	4	5	6	7
가공시간	6	4	9	3	2	7	1

- 작업의 순서? $\bar{F} = ?$ (1x6 + 2x5 + 3x4 + 4x3 + 6x2 + 7x1)

작업	7	5	4	2	1	6	3
가공시간	1	2	3	4	6	7	9
F	1	3	6	10	16	23	32

$$\frac{91}{7} = 13$$

Weighted Shortest Processing Time (WSPT) 규칙 (1/5)

WSPT 규칙은 SPT 규칙의 확장된 방법으로 각 작업마다 가중치 α 가 있다고 가정하고 작업의 가중 총 체류 시간 $\sum_{i=1}^n \alpha_i F_i$ 을 최소화 하기 위한 기법 (for $1 \leq i \leq n$)

- n 개의 작업이 있는 단일기계 환경에서 작업의 가중 총 체류시간 $\sum_{i=1}^n \alpha_i F_i$ 을 최소화하기 위한 WSPT 규칙은 다음과 같은 순서대로 작업을 배열하여 스케줄 한다.

$$\frac{p_{i(1)}}{\alpha_{i(1)}} \leq \frac{p_{i(2)}}{\alpha_{i(2)}} \leq \dots \leq \frac{p_{i(n)}}{\alpha_{i(n)}} \quad \text{p가 작을수록 } \alpha \text{가 클수록}$$

- 작업 i 의 가중체류시간 $\alpha_i F_i$ 에서 F_i 은 대기시간과 가공시간의 합이므로 재공시간이라고도 간주할 수 있다.
- 이러한 맥락에서 가중치 α_i 는 작업 i 의 단위 재공 체류 시간당 소요되는 비용 또는 작업의 시급성이라고 볼 수 있다.
- 그러므로 WSPT 규칙은 반도체 제조 공장처럼 재공이 오래 공장 내에서 머물면 안 좋은 작업에 큰 가중치(비용)를 부여하여 일정계획을 만 들고자 할 때 사용하는 기법이다.

Weighted Shortest Processing Time (WSPT) 규칙 (2/5)

정리 4.2

n 개의 작업이 있는 단일기계 환경에서 작업의 가중 총 체류시간($\sum_{i=1}^n \alpha_i F_i$)을 최소화하면 (또는 하려면) 다음 조건을 만족한다.

$$\frac{p_{(1)}}{\alpha_{(1)}} \leq \frac{p_{(2)}}{\alpha_{(2)}} \leq \dots \leq \frac{p_{(n)}}{\alpha_{(n)}} \rightarrow \text{WSPT}$$

(증명 1)

- 작업의 총 가중체류시간의 합인 $\sum_{i=1}^n \alpha_i F_i$ 는 다음과 같이 표현된다.

$$\sum_{i=1}^n \alpha_i F_i = \sum_{k=1}^n (\alpha_{(k)} W_{(k)} + \alpha_{(k)} p_{(k)}) = \sum_{k=1}^n \alpha_{(k)} W_{(k)} + \sum_{k=1}^n \alpha_{(k)} p_{(k)} \quad (4)$$

- 식 (4)에서 $\sum_{k=1}^n \alpha_{(k)} p_{(k)}$ 는 가중가공시간의 합으로 상수이다. 따라서 $\sum_{i=1}^n \alpha_i F_i$ 를 줄이려면 $\sum_{k=1}^n \alpha_{(k)} W_{(k)}$ 를 줄여야 한다. 가중대기시간을 스케줄에 있는 작업의 순서대로 다시 쓰면 다음과 같다.

Weighted Shortest Processing Time (WSPT) 규칙 (3/5)

정리 4.2 (continued)

- 가중대기시간을 스케줄에 있는 작업의 순서대로 다시 쓰면 다음과 같다.

$$\alpha_{(1)} W_{(1)} = 0$$

$$\alpha_{(2)} W_{(2)} = \alpha_{(2)} p_{(1)}$$

$$\alpha_{(3)} W_{(3)} = \alpha_{(3)} p_{(1)} + \alpha_{(3)} p_{(2)}$$

$$\vdots$$

$$\alpha_{(k)} W_{(k)} = \alpha_{(k)} p_{(1)} + \alpha_{(k)} p_{(2)} + \cdots + \alpha_{(k)} p_{(k-1)}$$

$$\vdots$$

$$\alpha_{(n)} W_{(n)} = \alpha_{(n)} p_{(1)} + \alpha_{(n)} p_{(2)} + \cdots + \alpha_{(n)} p_{(k-1)} + \cdots + \alpha_{(n)} p_{(n-1)}$$

(5)

Weighted Shortest Processing Time(WSPT) 규칙 (4/5)

정리 4.2 (continued)

- 식 (5)를 그림으로 표현하면 다음과 같다.

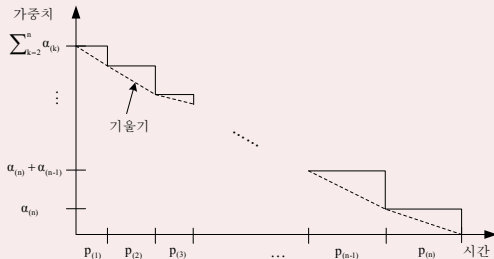


Figure 2: 총 가중체류시간

- 위의 그림에서 전체 면적인 총 가중체류시간을 최소화하기 위해서는 좌표상에 있는 점 $(0, \sum_{k=2}^n \alpha_{(k)})$ 와 점 $(\sum_{k=1}^n p_{(k)}, 0)$ 을 잇는 선분들의 기울기인 $-\alpha_{(k)}/p_{(k)}$ 가 가장 가파르도록 해야 하며, 이는 곧 $p_{(k)}/\alpha_{(k)}$ 가 가장 작은 작업부터 스케줄 되어야 한다는 것을 의미한다.

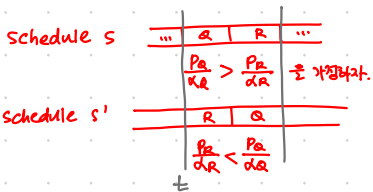
Weighted Shortest Processing Time (WSPT) 규칙 (5/5)

정리 4.2 (continued)

(증명 2) 모순 증명법 (Proof by contradiction)을 이용

- 최적 스케줄 S 가 $p_{(1)}/\alpha_{(1)} \leq p_{(2)}/\alpha_{(2)} \leq \dots \leq p_{(n)}/\alpha_{(n)}$ 조건을 만족하지 않는다고 가정하자. 그러면 스케줄상 의 연속된 위치에 있는 두 작업 Q 와 R 이 있고 $p_Q/\alpha_Q > p_R/\alpha_R$ 조건을 만족할 것이다.
- 이제 스케줄 S' 를 이 작업 Q 와 R 의 순서를 바꾼 순서라고 하자(다른 작업의 순서는 그대로 있다고 가정). 그리고 스케줄 S 에서 작업 Q 의 가공이 시작되는 시점을 t 라고 하자(시점 t 는 스케줄 S' 에서 작업 R 이 시작되는 시점).
- 위의 가정에 따라
 - (1) 스케줄 S 의 작업 Q 와 R 의 가중체류시간의 합: $\alpha_Q(t + p_Q) + \alpha_R(t + p_Q + p_R)$
 - (2) 스케줄 S' 의 작업 R 와 Q 의 가중체류시간의 합: $\alpha_R(t + p_R) + \alpha_Q(t + p_R + p_Q)$
- 작업 Q 와 R 를 제외한 다른 작업의 가중체류시간은 변화가 없으므로 스케줄 S 의 총 가중체류시간에서 스케줄 S' 의 총 가중체류시간을 빼면 $\alpha_R p_Q - \alpha_Q p_R$ 이 되는데 이는 가정에 의해 0 보다 크다. 따라서 스케줄 S' 의 총 가중체류시간이 S 보다 작다.
- 이런 증명 과정을 반복적으로 적용하면 $p_{(1)}/\alpha_{(1)} \leq p_{(2)}/\alpha_{(2)} \leq \dots \leq p_{(n)}/\alpha_{(n)}$ 이 $\sum_{i=1}^n \alpha_i F_i$ 을 최소화하는 스케줄의 조건이라는 것을 알 수 있다. Q.E.D.

$$\frac{p(1)}{\alpha(1)} \leq \frac{p(2)}{\alpha(2)} \leq \dots \leq \frac{p(n)}{\alpha(n)}$$



$$F_i = C_i - \checkmark$$

$$s \rightarrow (t + p_\alpha) w_\alpha + (t + p_\alpha + p_R) w_R \rightarrow p_\alpha w_R$$

$$s' \rightarrow (t + p_R) w_R + (t + p_R + p_\alpha) w_\alpha \rightarrow p_R w_\alpha$$

$$s \rightarrow \frac{p_\alpha}{w_\alpha} > \frac{p_R}{w_R} \leftarrow s'$$

Earliest Due Date (EDD) 규칙 (1/4) 납기

EDD 규칙은 단일기계 환경에서 작업의 납기로부터 가장 늦은 작업의 지연시간($L_{\max}$)을 최소화 하기 위한 스케줄링 기법 (for $1 \mid \mid L_{\max}$) $\rightarrow T_{\max}$

- 작업들을 납기의 오름차순으로 배열하여 스케줄 한다.

$$d_{i(1)} \leq d_{i(2)} \leq \dots \leq d_{i(n)}$$

여기서 $d_{i(k)}$ 는 k 번째 스케줄된 작업 J_i 의 납기이다.

- 즉 EDD 규칙은 $(1 \mid \mid L_{\max})$ 문제를 풀기 위한 방법이며, $L_{\max}$ 를 줄이면 $T_{\max}$ 도 함께 줄기 때문에 EDD rule 은 $1 \mid \mid T_{\max}$ 에 대한 최적 스케줄도 제공한다.

Earliest Due Date (EDD) 규칙 (2/4)

정리 4.3

n 개의 작업이 있는 단일기계 환경에서 작업의 최대 지연시간($L_{\max}$)을 최소화하면 (또는 하려면) 다음 조건을 만족한다.

$$d_{i(1)} \leq d_{i(2)} \leq \dots \leq d_{i(n)}$$

(증명) 모순 증명법 (Proof by contradiction)

- 다음과 같이 납기의 오름차순으로 배열하지 않은 스케줄 S 를 정의하자.
 - $d_{i(k)} > d_{i(k+1)}$ 되도록 $k, k+1$ 번째 가공되는 작업을 배열한다.
 - 편의상 k 번째로 가공되는 작업 $J_{i(k)}$ 를 A 그리고 $k+1$ 번째 가공되는 작업 $J_{i(k+1)}$ 를 B 라고 하면, $d_A > d_B$ 이 된다.
- 위에서 정의한 스케줄 S 에서 작업 A 와 B 의 순서를 바꾼 스케줄을 S' 으로 정의하자.
 - 이 경우 스케줄 S 에서 A 와 B 를 제외한 $(n-2)$ 개의 작업들의 Lateness 중에 가장 큰 값을 L 이라 하고 스케줄을 S' 에서 그와 같은 의미를 갖는 값을 L' 이라 하면 $L = L'$ 이 되고, 정의에 의해서 $L_{\max}$ 와 $L'_{\max}$ 를 다음과 같이 정리할 수 있다.

$$\begin{aligned} L_{\max} &= \max\{L, L_A, L_B\} \\ L'_{\max} &= \max\{L', L'_A, L'_B\} = \max\{L, L'_A, L'_B\} \end{aligned} \quad (6)$$

schedule s

...	K	K+1	...
-----	---	-----	-----

 $d_k > d_{k+1}$

schedule s'

...	K+1	K	...
-----	-----	---	-----

 $d_{k+1} < d_k$

$$\text{for } s, \quad L_i(k) = C_i - d_i = \sum_{j=1}^{k-1} P_{i(j)} + P_{i(k)} - d_{i(k)} \quad \textcircled{A}$$

$$L_i(k+1) = \sum_{j=1}^k P_{i(j)} + P_{i(k+1)} - d_{i(k+1)} \quad \textcircled{B}$$

$$\text{for } s', \quad L_i(k+1) = \sum_{j=1}^{k-1} P_{i(j)} + P_{i(k+1)} - d_{i(k+1)} \quad \textcircled{C}$$

$$L_i(k) = \left(\sum_{j=1}^{k-1} P_{i(j)} + P_{i(k+1)} + P_{i(k)} - d_{i(k)} \right) \quad \textcircled{D}$$

$$\max\{A, B\} \geq \max\{C, D\} \text{ 성립.}$$

$$\begin{aligned} \because B &\geq C \dots P_{i(k)} \cancel{\text{가}} \\ \because B &\geq D \dots d_{i(k)} - d_{i(k+1)} \cancel{\text{가}} \end{aligned} \quad \left. \vphantom{\begin{aligned} \because B &\geq C \dots P_{i(k)} \cancel{\text{가}} \\ \because B &\geq D \dots d_{i(k)} - d_{i(k+1)} \cancel{\text{가}} \end{aligned}} \right\} \text{ 성립.}$$

Earliest Due Date (EDD) 규칙 (3/4)

정리 4.3 (continued)

- 스케줄 S 와 S' 에 대해 L_A, L_B , 그리고 L'_A, L'_B 를 구체적으로 표현하면 다음과 같다.

$a = \sum_{\ell=1}^{k-1} p_{(\ell)}$ 라고 정의하면,

- 스케줄 S 의 경우 ($A \rightarrow B$ where $d_A > d_B$ not following EDD)

$$L_A = a + p_A - d_A \quad (a = \sum_{\ell=1}^{k-1} p_{i(\ell)}), \quad L_B = a + p_A + p_B - d_B$$

- 스케줄 S' 의 경우 ($B \rightarrow A$ where $d_A > d_B$ following EDD)

$$L'_B = a + p_B - d_B \quad (a = \sum_{\ell=1}^{k-1} p_{i(\ell)}), \quad L'_A = a + p_A + p_B - d_A$$

- $p_A > 0$ 이므로 $L_B > L'_B$, 그리고 $d_A > d_B$ 이므로 $L_B > L'_A$ 관계 성립한다. 따라서,

$$L_B > \max\{L'_A, L'_B\} \quad (7)$$

- 식(6)과 (7)를 이용하면

$L_{\max} = \max\{L, L_A, L_B\} \geq \max\{L, L_B\} \geq \max\{L, L'_A, L'_B\} = L'_{\max}$ 가 성립한다.

그러므로 어떤 스케줄이든 납기의 오름차순으로 재정렬 한다면 $L_{\max}$ 를 증가시키지 않는다. Q.E.D.

Earliest Due Date (EDD) 규칙 (4/4)

예제 4.2

다음의 $(1 \mid \mid L_{\max})$ 문제를 풀어보자.

작업	1	2	3	4	5	6
납기	7	3	8	12	9	3
가공시간	1	1	2	4	1	3

- 작업의 순서? $L_{\max} = ?$

Job Sequence	Processing Time (p_j)	Completion Time (C_j)	Due Date (d_j)	Lateness ($L_j = C_j - d_j$)
2	1	1	3	1 - 3 = -2
6	3	1 + 3 = 4	3	4 - 3 = 1
1	1	4 + 1 = 5	7	5 - 7 = -2
3	2	5 + 2 = 7	8	7 - 8 = -1
5	1	7 + 1 = 8	9	8 - 9 = -1
4	4	8 + 4 = 12	12	12 - 12 = 0

Moore 알고리즘 (1/2)

of tardy jobs

Moore의 알고리즘은 단일기계 환경에서 납기를 초과한 작업의 수 인 n_T 를 최소화 하기 위한 스케줄링 기법 (for $1 \parallel n_T$)

- Moore 알고리즘은 기본적으로 EDD 규칙을 기반으로 하며 다음과 같이 4 단계로 구성된다.

Step 1. EDD규칙에 따라 작업을 정렬 한 후 다음 조건을 만족하는 현재 스케줄을 찾는다. 따라서 현재 스케줄 $(J_{(1)}, J_{(2)}, \dots, J_{(n)})$ 는 $d_{(i)} \leq d_{(i+1)}, i = 1, 2, \dots, n-1$ 를 만족한다.

Step 2. 현재 스케줄 에서 첫 번째 지연 작업 $J_{(k)}$ 를 찾는다. 만약 지연된 작업이 없다면 Step 4로 이동한다.

Step 3. 부분 스케줄 $(J_{(1)}, J_{(2)}, \dots, J_{(k)})$ 에서 가공시간이 가장 큰 작업을 제거하고 Step 2로 되돌아간다. 현재 스케줄이 전 보다 하나 짧아 진다.

Step 4. 현재 스케줄 의 뒤에 Step 3에서 제거된 작업을 붙여서 최적 스케줄을 얻는다 (제거된 작업의 순서는 상관이 없다).

Moore 알고리즘 (2/2)

예제 4.3

다음의 $(1 | n_T)$ 문제를 풀어보자.

작업	1	2	3	4	5	6
납기	15	6	9	23	20	30
가공시간	10	3	4	8	10	6

- 작업의 순서? $n_T = ?$

J 2 3 1 5 4 6
 C 3 7 17 27 35 41
 d 6 9 15 20 23 30

J 2 3 5 4 6
 C 3 7 17 25 31
 d 6 9 20 23 30

$[2-3-4-6-(1,5)]$

Lawler 알고리즘 (1/4)

$\alpha \beta \gamma$



precedence

Lawler 알고리즘은 작업 선행 순서가 있을 때 각 작업 i 의 완료시점 C_i 의 함수 $f_i(C_i)$ 중에서 최대값을 최소화 하기 위한 스케줄링 기법 (for $1 \leq i \leq \max_{i=1}^n \{f_i(C_i)\}$)

- 일부 작업간에는 실행 순서가 미리 정해져 있는 선행조건이 존재할 수 있다. 이 조건은 반드시 지켜져야 하며, 선행 조건이 없는 작업들은 일정계획 알고리즘에 따라 순서가 만들어 질 수 있다. 작업선행 조건 이 많아지면 고려해야 할 스케줄의 수가 줄어든다는 이점이 있다.
- 여기서 $f_i(C_i)$ 는 C_i 에 대한 비감소 함수인 Regular measure 라는 특성을 만족해야 한다. 이를 최적화 문제로 표현하면 다음과 같다.

$$\begin{aligned} & \text{Minimize} \quad \max_{i=1}^n \{f_i(C_i)\} \\ & \text{subject to} \quad \text{작업선행조건} \end{aligned}$$

- 만일 $f_i(C_i) = C_i - d_i$ 이면 성능지표는 $\max_{i=1}^n (L_i) = L_{\max}$ 가 되고 $f_i(C_i) = \max\{C_i - d_i, 0\}$ 이면 성능지표는 $\max_{i=1}^n (T_i) = T_{\max}$ 가 된다. 그리고 이 알고리즘은 작업 선행 순서가 없을 때도 적용 가능하므로 매우 강력한 단일기계 일정계획 알고리즘이다.

Lawler 알고리즘 (2/4)

- Lawler 알고리즘의 단계는 다음과 같다.

Step 1. 스케줄 할 수 있는 모든 작업의 총 작업완료시점 $\tau = \sum_{i=1}^n p_i$ 를 구한다. 그리고 스케줄의 끝에 위치할 수 있는 작업의 집합 V 를 구성한다. $|V| = 1$ 이면 종료한다.

Step 2. $f_k(\tau) = \min_{J_i \in V} \{f_i(\tau)\}$ 인 작업 J_k 를 발견하여 스케줄의 맨 뒤에 위치시킨다. $V = V - \{J_k\}$ 로 변경하고 Step 2으로 돌아간다.

$$\min_{i \in V} f_i(c_i)$$



$$c_i \rightarrow \tau$$

$$\tau = (\sum p_i - p_{i(c_i)})$$

가장 마지막에 수행되는 작업의 종료시점.

Lawler 알고리즘 (3/4)

정리 4.4

$1|prec|\max_{i=1}^n \{f_i(C_i)\}$ 일정계획 문제에서 V 를 스케줄 마지막에 위치할 수 있는 작업의 집합이라고 하자. 즉 V 에 속한 작업은 다른 작업보다 선행한다는 조건이 없다. 그리고 J_k 를 V 에 속하고 $f_k(\sum_{i=1}^n p_i) = \min_{J_i \in V} \{f_i(\sum_{i=1}^n p_i)\}$ 을 만족하는 작업이라고 하자. 그러면 최적 스케줄에는 작업 J_k 가 마지막 위치에 존재하게 된다.

(증명)

- 최적 스케줄 S 가 (A, J_k, B, J_ℓ) 이고 여기서는 작업 J_k 가 스케줄의 마지막 위치에 존재하지 않는다고 가정하자. 그리고 스케줄 S' 은 (A, B, J_ℓ, J_k) 이라고 하자. 그러면 S 는 작업 선행 순서를 만족하고 J_k 는 스케줄의 마지막 위치에 올 수 있으므로 S' 도 가능한(Feasible)한 스케줄이다.
- 또한 모든 작업 i 에 대해서 $f_i(C_i)$ 는 C_i 에 대한 비감소 함수이고 J_k 를 제외한 모든 작업 i 의 완료시점은 스케줄이 S 에서 S' 으로 변환에 따라 증가하지 않으므로 J_k 를 제외하고는 추가 비용이 발생하지 않는다.
- J_k 는 $f_k(\sum_{i=1}^n p_i) = \min_{J_i \in V} \{f_i(\sum_{i=1}^n p_i)\}$ 특성을 가지고 있기 때문에 $f_k(\sum_{i=1}^n p_i) \leq f_\ell(\sum_{i=1}^n p_i)$ 을 만족한다.
- 스케줄이 S 에서 S' 으로 변하더라도 S' 에서 J_k 로 인하여 추가되는 비용은 S 에서 J_ℓ 에 의해 발생하는 비용보다 적다. 그러므로 S' 가 S 보다 좋은 해이며 J_k 는 스케줄의 마지막 위치에 존재해야 한다. Q.E.D.

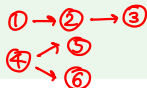
Lawler 알고리즘 (4/4)

예제 4.4

다음의 $(1|prec|L_{\max})$ 문제를 풀어보자.

작업	1	2	3	4	5	6
납기	3	6	9	7	11	7
가공시간	2	3	4	3	2	1

작업 선행순서 조건: $J_1 \rightarrow J_2 \rightarrow J_3, J_4 \rightarrow J_5, J_4 \rightarrow J_6$



- 작업의 순서? $L_{\max} = ?$

$$\tau = 15 \quad V = \{3, 5, 6\} \quad f_3(\tau) = 15 - 9 = 6$$

$$f_5(\tau) = 15 - 11 = 4 \rightarrow \textcircled{5} \text{ 맨뒤로!}$$

$$f_6(\tau) = 15 - 7 = 8$$

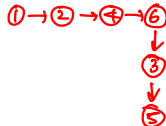
$$\tau = 15 - 2 = 13 \quad V = \{3, 6\} \quad f_3(\tau) = 13 - 9 = 4 \rightarrow \textcircled{3} \text{ 맨뒤로!}$$

$$f_6(\tau) = 13 - 7 = 6$$

$$\tau = 13 - 4 = 9 \quad V = \{2, 6\} \quad f_2(\tau) = 9 - 6 = 3$$

$$f_6(\tau) = 9 - 7 = 2 \rightarrow \textcircled{6} \text{ 맨뒤로!}$$

(반복)



건설적 알고리즘 종합

관찰사항 4.1

단일기계 문제에서 $r_i = 0$ 이면 비지연 스케줄 집합에 최적 스케줄이 존재하게 된다.

- 지금까지 알아본 일정계획 알고리즘은 모두 작업의 도착시간이 0 이라는 가정이 있었다. SPT 규칙, WSPT 규칙, EDD 규칙, Moore 알고리즘, Lawler 알고리즘 모두 성능지표는 다르지만 성능지표를 최소화하기 위해 고의적으로 기계의 유휴시간을 삽입하지 않는 비지연(Non delay) 스케줄을 만든다는 공통점이 존재
- 하지만 $r_i \neq 0$ 이면 최적 스케줄은 비지연 스케줄 집합에 속하지 않는 단일기계 일정계획 문제가 꽤 존재한다. 다음의 $1|r_i|\sum_{i=1}^n w_i C_i$ 문제를 살펴보자.

작업	1	2
가공시간	1	3
도착시간	1	0
가중치	2	1

Table 1: 작업별 가공시간, 도착시간, 가중치

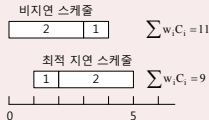


Figure 3: 총 가중작업완료시간과 스케줄

- 위의 그림에서 보듯이 비지연 스케줄보다 활성이지만 지연인 스케줄이 더 좋은 결과를 가져온다. 사실 $r_i \neq 0$ 인 문제는 단일기계 환경일지라도 NP-hard라서 최적의 스케줄을 찾자 한다면 현재까지 Exponential 계산 복잡도를 갖는 알고리즘밖에 없다.

암묵적 열거법 (1/3) $\rightarrow$ implicit $\leftrightarrow$ explicit

EDD 규칙과 Lawler 알고리즘은 납기와 관련된 단일기계 일정계획 문제에서 성능지표가 Min-Max 형태일 때 아래와 같은 증명과정을 통해서 최적임을 보였다.

- 증명과정 *by contradiction* : $S \rightarrow S' \rightarrow \dots \rightarrow S^*$
 $\xrightarrow{\text{해 개선}}$
 (1) 스케줄 S 가 주어진 조건을 만족하지 않았다고 가정하고
 (2) 조건을 만족시키도록 작업을 한 번에 하나씩 이동시킨 이웃 스케줄 S' 을 만들고
 (3) S' 이 S 보다 성능지표 값이 좋다고 증명한 후
 (4) 조건이 모두 만족할 때까지 나머지 작업을 지속적으로 이동시키면 최적 스케줄 S^* 에 도달한다.
- 주목해야 할 점은 이 증명 과정은 조건을 만족하는 이웃 스케줄은 항상 성능지표가 개선된다는 것이다. 그렇다면 납기와 관련된 문제에서 성능지표가 Total/Average 인 경우에도 같은 증명 방식을 할 수 있을까?

암묵적 열거법 (2/3)

예제 4.5

아래 표와 그림은 3개의 작업에 대한 모든 이웃 스케줄과 총 작업 순수지연시간을 보여준다.

작업	1	2	3
가공시간	1	2	3
납기	4	2	3

Table 2: 작업의 가공시간과 납기

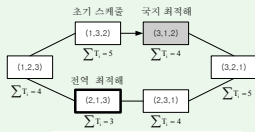


Figure 4: 이웃 스케줄과 성능지표

- 이 그림에서 초기 스케줄을 (1, 3, 2)라고 가정하자. 이웃 스케줄인 (3, 1, 2)와 (1, 2, 3)은 모두 성능지표 값이 같으므로 (3, 1, 2)를 선택해 보자.
- 그러면 새로운 스케줄 (3, 1, 2)은 이웃인 (1, 3, 2)와 (3, 2, 1)보다 성능이 좋으므로 국지 최적해(Local optimum)이고 따라서 탐색은 중단 될 것이다.
- 그러나 전역 최적 해는 스케줄 (2, 1, 3)에 있다. 여기서 초기 스케줄 (1, 3, 2)에 비해서 이웃 스케줄인 (3, 1, 2)와 (1, 2, 3)은 모두 성능지표 값이 개선되었고 같으므로 어느 쪽을 선택해야 할지에 대한 조건을 만들기가 어렵다는 점이다.
- 이 예제는 성능지표가 Total Average인 경우 작업의 위치 교환만을 통해서 항상 전역 최적 해를 보장할 수 없다는 것을 보여준다.

암묵적 열거법 (3/3)

관찰사항 4.2

단일기계 문제에서 $r_i = 0$ 일지라도 납기와 관련된 Total(Average) 형태의 성능지표 T 가 목적함수에 존재하면 성능을 지속적으로 개선하는 이웃 해 조건을 알아내기가 매우 어렵다. 즉 이웃해 탐색방법은 전역 최적 해를 보장하질 못한다.

- 성능지표 T 를 고려하는 일정계획 문제는 단일기계일지라도 NP-hard에 속한다.
- 따라서 작업의 수가 많아지면 최적 스케줄을 적절한 시간 내에 찾기는 매우 어렵다. 그렇다고 최적 스케줄을 발견하는 알고리즘이 없는 것은 아니다.
- 이제부터는 성능지표가 T 인 일정계획 문제도 풀 수 있는 동적 계획법과 분지 한계법을 소개한다.
- 두 알고리즘은 일정계획 문제 뿐만 아니라 일반적인 정수 계획 문제에도 적용이 가능하다는 장점이 있다. 그러나 문제 크기가 조금만 커지면 최적 해를 찾아내는데 너무나 오랜 시간이 걸리는 Exponential 계산 복잡도를 갖는다는 단점이 있다.

동적 계획법(Dynamic Programming) (1/8)

동적 계획법은 선형계획 문제나 정수계획 문제에도 모두 적용 할 수 있는 대표적인 암묵적 열거법이며, 다음과 같은 독특한 방식으로 문제를 해결한다.

1. 최적화 문제를 여러 **단계(Stage)**로 구분한다.
2. 각 단계에서는 여러 **상태(State)**를 갖는다.
3. 단계의 각 상태 는 여러 의사결정 이 있으며, 의사결정마다 **보상(Reward)**이 주어진다.
4. 단계의 상태에서 최적 의사결정과 목적함수 값은 다음 **재귀(Recursive)**식으로 표현된다.

$$\left(\begin{array}{c} \text{단계 } j \text{의 상태 } S_j \text{에서의} \\ \text{최적 목적함수 값} \end{array} \right) = \min_{\left(\begin{array}{c} S_j \text{에서 가능한} \\ \text{모든 의사결정} \end{array} \right)} \left\{ \left(\begin{array}{c} \text{의사결정에 따른} \\ \text{보상값} \end{array} \right) + \left(\begin{array}{c} \text{단계 } j-1 \text{의 상태 } S_{j-1} \\ \text{에서의 최적 목적함수 값} \end{array} \right) \right\}$$

$\downarrow$
immediate reward
 $\downarrow$
이전 계산되어 있어야 함.

(8)

DP { forward : 단계 1부터 전개
 backward : 단계 n부터 전개



재귀식의 전개방향에 따라
 not transition 방향.

$\textcircled{j} \rightarrow x_j \rightarrow \textcircled{j-1}$: transition

동적 계획법(Dynamic Programming) (2/8)

- 재귀식 (8)에서 단계 $j-1$ 의 상태 S_{j-1} 는 아래 그림에서 보듯이 단계 j 의 상태 S_j 에서 내린 의사결정에 따라서 결정된다.
- 동적 계획법에서 단계는 시작 단계부터 마지막 단계까지 순차적으로 진행되나, 각 단계의 상태에서의 최적 의사결정은 이전 단계의 상태를 고려한다.

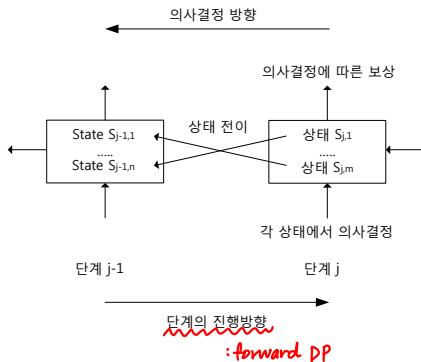


Figure 5: 단계와 상태

동적 계획법(Dynamic Programming) (3/8)

- 좀 더 수리적으로 재귀 식을 표현해 보자. 단계 j 에서 상태 S_j 의 의사결정을 k 라고 하고 이에 따른 보상을 $R(k)$ 라고 정의하자.
- 그리고 단계 j 에서 상태 S_j 의 최적 목적함수 값을 $f(S_j)$ 라고 하자. 그러면 재귀식 (8)은 다음과 같이 표현된다.

$$f(S_j) = \min \left(\begin{array}{l} \text{all feasible } k \text{ that} \\ \text{can be made at state } S_j \end{array} \right) \{R(k) + f(S_{j-1})\} \text{ for every state } S_j \text{ and stage } j$$

$(S_j \rightarrow x_j \rightarrow S_{j-1})$
 : state transition model. (9)

- 위의 재귀식 (9)을 가만히 살펴보면 비록 S_{j-1} 은 S_j 와 k 에 의해서 결정되지만 S_{j-1} 에서의 최적 목적함수 값인 $f(S_{j-1})$ 자체는 S_j 와 k 에 영향을 받지 않는다. 만일 영향을 받는다면 $f(S_{j-1})$ 를 $f(S_{j-1} : S_j, k)$ 로 표현해야 할 것이다.
- 다시 말하면 **현 단계의 의사결정이 그전 단계에 진행된 의사결정을 변경하지 않는다는 뜻으로 이를 독립성(Independence) 또는 최적원리(Principle of optimality)라고 부른다.** 그리고 최적 원리를 유지하려면 목적함수가 보상들의 합으로 구성되어야 한다. 이를 **합의성질(Additive property)**이라 부른다.

동적 계획법(Dynamic Programming) (4/8)

- 이제 일정계획 문제를 동적 계획법으로 풀어보기 위해 단일기계에서의 목적함수 Z 를 다음과 같이 각 작업 i 의 완료시점 C_i 의 함수 $g(C_i)$ 의 합이라고 생각해 보자.

$$Z = \sum_{i=1}^n g(C_i) \rightarrow \min Z$$

- $g(C_i)$ 의 형태에 따라 다음과 같이 다양한 목적함수를 구성할 수 있다.
 - $g(C_i)$ 가 $\max\{0, C_i - d_i\}$ 이면 목적 함수 Z 는 **순수지연시간의 합**이 된다.
 - $g(C_i) = w_i \delta[\max\{0, C_i - d_i\}]$ 이고 δ 를 Indicator함수, w_i 를 작업 i 의 가중치라고 하면 목적 함수 Z 는 **가중 순수지연 작업수가** 된다.
- n 개 작업의 일정계획을 위한 **단계, 상태, 의사결정, 보상**을 다음과 같이 정의

단계	작업의 부분집합 크기로 정의한다. 작업이 총 n 개 있으므로 부분집합 크기는 1부터 n 까지 존재한다. 즉 단계는 1부터 n 까지 존재한다. 스케줄 해에 해당하는 작업수
상태	작업의 부분집합으로 정의한다. $n = 3$ 이고 단계가 2이면 가능한 상태는 $\{1, 2\}, \{1, 3\}, \{2, 3\}$ 이다. 3개 중 2개를 스케줄 해줘야함
의사결정	스케줄의 마지막 위치하는 작업을 결정하는 것으로 정의한다. $n = 3$ 이고 단계가 2, 상태가 $\{2, 3\}$ 이면 마지막 작업은 작업 2 또는 작업 3이 될 수 있다.
보상	의사결정에 따른 $g(C_i)$ 값이다. $g(C_i)$ 가 $\max\{0, C_i - d_i\}$ 이라고 생각해 보자. $n = 3$ 이고 단계가 2, 상태가 $\{2, 3\}$, 의사결정인 마지막 작업을 작업 2라고 하면 보상은 $g(C_2)$ 즉 작업 2의 순수지연시간이 된다.

동적 계획법(Dynamic Programming) (5/8)

- 목적함수에 대한 재귀식을 예를 들어 정의해보면 다음과 같다. $n = 3$ 이고 단계가 2, 상태가 $\{2, 3\}$ 일 때 최적의 목적함수 값은 다음과 같이 표현된다.

$$f_2(\{2, 3\}) = \min_{i \in \{2, 3\}} [g(C_i) + \overset{\text{red arrow}}{f_1(\{2, 3\} - \{i\})}]$$

n 개 작업에 대한 재귀식은 다음과 같이 정의 된다.

- **단계 1:** 가능한 상태 S_1 은 $\{1\}, \{2\}, \dots, \{n\}$ 이 된다. 각 상태마다 재귀식은 다음과 같다.

$$f_1(S_1) = \min_{i \in S_1} [g(C_i) + \overset{\text{red arrow}}{f_0(\cdot)}] = \min_{i \in S_1} [g(C_i)]$$

- **단계 j ($2 \leq j \leq n$):** 가능한 상태 S_j 은 총 작업 n 개 중에 j 개 작업으로 구성된 부분집합들이 된다. 각 상태마다 재귀식은 다음과 같다.

$$f_j(S_j) = \min_{i \in S_j} [g(C_i) + f_{j-1}(S_j - \{i\})]$$

동적 계획법(Dynamic Programming) (6/8)

예제 4.6

다음의 $(1 | \sum_{i=1}^n T_i)$ 문제를 풀어보자.

작업	1	2	3	4
가공시간	5	6	9	8
납기	9	7	11	13

[forward]

단계 1

s_1	$\{1\}$	$\{2\}$	$\{3\}$	$\{4\}$
$i \in s_1$	1	2	3	4
$g(c_i)$	0	0	0	0
$f_0(\emptyset)$	0	0	0	0
$f_1(s_i)$	0	0	0	0

$\downarrow$
 $\min \{ g(c_i) + f_0(s_1 - \{i\}) \}$

단계 2

s_2	$\{1,2\}$	$\{1,3\}$	$\{1,4\}$	$\{2,3\}$	$\{2,4\}$	$\{3,4\}$
$i \in s_2$	1, 2	1, 3	1, 4	2, 3	2, 4	3, 4
$g(c_i)$	2	4	5	3	4	6
$f_1(s_2 - \{i\})$	0	0	0	0	0	0
$f_2(s_2)$	2	3	0	4	1	4

Handwritten notes: $s: 2 \rightarrow 1$, $11-9=2$, $11-1=10$, $14-9=5$, $14-1=13$. Arrows indicate the calculation of $f_2(s_2)$ from $f_1(s_2 - \{i\})$ and $g(c_i)$.

उदा 3	S_3	{1,2,3}	{1,2,4}	{1,3,4}	{2,3,4}							
$i \in S_3$	1	2	3	1	2	3	4					
$g(C_i)$	11	13	9	10	12	6	13	11	9	16	12	10
$f_2(S_3 - \{i\})$	4	3	2	1	0	2	4	0	3	4	1	4
				11		8		11			13	

उदा 4	S_4	1. 2. 3. 4		
$i \in S_4$	1	2	3	4
$g(C_i)$	28-9 =19	28-7 =21	28-11 =17	28-13 =15
$f_3(S_4 - \{i\})$	13	11	8	11
$f_4(S_4)$			25	

$$f_4(S_4) = \min \{ g(C_i) + f_3(S_4 - \{i\}) \}$$

$$2 \rightarrow 1 \rightarrow 4 \rightarrow 3$$

동적 계획법(Dynamic Programming) (7/8)

Forward and Backward Dynamic Programming

- 앞의 예제에서와 같이 재귀식을 단계 1부터 단계 n 으로 만들어가는 방식을 선행전개 동적 계획법(Forward dynamic programming) 이라고 부른다. 선행전개 동적 계획법에서는 최적 해를 단계 n 에서부터 단계 1로 찾아간다.
- 반대로 재귀 식을 단계 n 부터 단계 1로 만들어가는 방식을 후행전개 동적계획법(Backward dynamic programming) 이라고 부르며 이 방법에서는 최적 해를 단계 1에서부터 단계 n 으로 찾아간다.

단계 4. S_4 {1, 2, 3, 4}

$$f_4(S_4) = 0$$

Backward DP

Job	1	2	3	4
p	5	6	9	8
d	9	7	11	13

단계 3 S_3 {1, 2, 3} {1, 2, 4} {1, 3, 4} {2, 3, 4}

$i \in S_3$	4	3	2	1
$g(S, i)$	15	17	21	19
$f_4(S_4 + i)$	0	0	0	0
$f_3(S_3)$	15	17	21	19

단계 2 S_2 {1, 2} {1, 3} {1, 4} {2, 3} {2, 4} {3, 4}

$i \in S_2$	3	4	2	4	2	3	1	4	1	3	1	2
$g(S, i)$	9	6	13	9	12	11	9	10	10	12	13	16
$f_3(S_3 + i)$	15	17	15	21	17	21	15	19	17	19	21	19
$f_2(S_2)$		23	28		29		24		27		34	

간격	S_1	$\{1\}$				$\{2\}$				$\{3\}$				$\{4\}$			
$i \notin S_1$		2	3	4	1	3	4	1	2	4	1	2	3				
$g(S, i)$		4	3	0	2	4	1	5	8	4	4	1	6				
$f_2\{S_1 + \{i\}\}$		23	28	29	23	24	21	28	24	34	29	21	34				
$f_1(S_1)$		21			25				32		33						

$$2 - 1 - 4 - 3$$

$n \neq 3 \nmid 13 \nmid 27$.

동적 계획법(Dynamic Programming) (8/8)

차원의 저주(Curse of Dimensionality)

동적 계획법은 작업의 수 n 이 증가할수록 대략 2^n 개 만큼의 상태(작업 부분집합)에 대해서 재귀 식을 계산해야 한다.

- 이런 단점 때문에 알고리즘의 일반성이라는 장점에도 불구하고 동적 계획법은 자주 사용되는 최적화 방법은 아니다.
- 변수의 수가 조금 증가하더라도 계산량은 급격히 증가하는 것을 빗대어 **차원의 저주**라고 부른다.
- 그러나 동적 계획법은 작업의 모든 스케줄을 열거하고 이 중에서 가장 좋은 스케줄을 발견하는 단순열거법(Explicit enumeration)보다는 효율적이다. 작업의 수 n 일 때 단순 열거법은 $n!$ 개 만큼의 스케줄을 고려해야 하나 동적 계획법은 이보다 적은 대략 $n2^n$ 개 정도의 스케줄만 고려하면 된다.

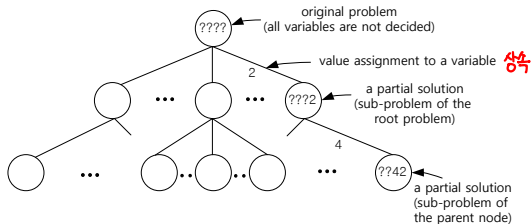
왜 $O(n2^n)$ 이냐....

분지 한계법(Branch and Bound) (1/18)

분지 한계법은 정수 계획 문제를 풀기 위한 대표적인 알고리즘이며, 암묵적 열거법이다. 분지 한계법에서는 **탐색 트리(Search tree)**를 이용해서 해를 찾아간다.

- 노드 (Node):** 한 개의 부분 해 (Partial Solution) 또는 부분 문제 (Sub problem) 을 의미 한다. 예를 들어 아래 그림 의 노드(???2) 는 스케줄의 마지막 위치에 작업 2를 배정한 것으로 부분 해이며, 아직 첫 번째부터 세 번째 위치의 작업을 결정 안 했기 때문에 부분 문제이기도 한다. 즉 루트(Root) 노드와 맨 하위 노드를 제외한 노드는 작업이 스케줄의 마지막 위치부터 배치된 부분 해이며, 각 부분 해는 아직 배치되지 않은 작업이 있으므로 부분 문제이기도 하다. 루트 노드에는 모든 작업이 스케줄 되어 있지 않으며 맨 하위 노드는 스케줄의 마지막부터 시작 위치까지 모든 작업이 배치 되었으므로 실행 가능 해가 된다.

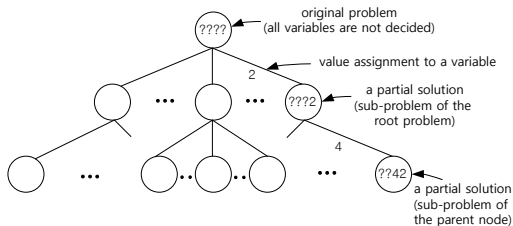
부분해



분지 한계법(Branch and Bound) (2/18)

분지 한계법은 정수 계획 문제를 풀기 위한 대표적인 알고리즘이며, 암묵적 열거법이다. 분지 한계법에서는 탐색 트리(Search tree)를 이용해서 해를 찾아간다.

- 분지 (Branching):** 부모 노드에서 결정되지 않은 변수에 값을 할당하여 자식 노드를 생성하는 과정이다. 아래 그림에서 분지는 루트 노드 에 작업 2를 스케줄의 마지막 위치에 할당하는 것에 해당한다. 이 분지를 통해서 자식노드가 생성되며 부분 해 (???2)가 만들어 진다.



Branching

4.

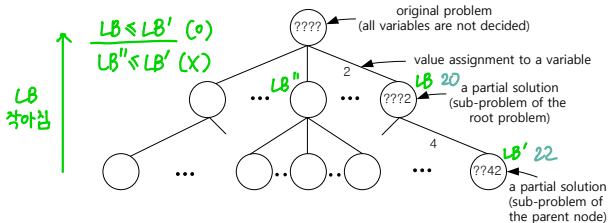
마지막 단계.

Figure 6: 탐색 트리

분지 한계법(Branch and Bound) (3/18)

분지 한계법은 정수 계획 문제를 풀기 위한 대표적인 알고리즘이며, 암묵적 열거법이다. 분지 한계법에서는 탐색 트리(Search tree)를 이용해서 해를 찾아간다.

- 탐색 트리에서 모든 작업이 스케줄에 배치될 때까지 분지를 계속 하면 결국 맨 하위 노드들은 실행가능해가 될 것이며 이런 방법은 암묵적이 아닌 단순 열거법이다. 따라서 분지 한계법에서는 한계를 통하여 암묵적 열거를 실행한다. *minimization 문제*
- 한계 (Bounding):** 노드의 부분 문제에서 하한 경계 값 (Lower bound)을 계산하고 노드 아래로 분지를 할 것인지 아닌지를 결정하는 과정이다.



???2 의 노드 T의 min보다 더 작은 LB.
T > LB

부분해의 LB는 크게 찾는수록 좋다. $\Rightarrow$ higher LB.

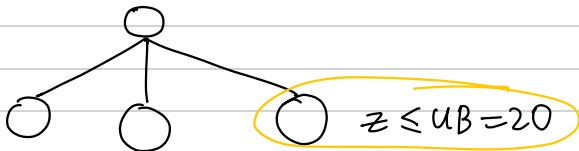
↓
재밌는 것인가!

Figure 6: 탐색 트리

[1, 2, 3, 4] LB = 15 라면 ???2 노드 탐색할 필요 없음.

LB = 25 라면 ???2 노드 탐색해야함.

Max 문제
Upper Bound.



지금까지 best

$$\bar{z} = 15$$



◎ 안일치야해

$$25$$



◎ 일치돼서.

가장 작은 UB를 찾아야함!

분지 한계법(Branch and Bound) (4/18)

분지 한계법은 정수 계획 문제를 풀기 위한 대표적인 알고리즘이며, 암묵적 열거법이다. 분지 한계법에서는 탐색 트리(Search tree)를 이용해서 해를 찾아간다.

- **경계 값(Lower & Upper Bound):** 각 노드에서 풀어야 할 부분 문제에서 제약을 하나이상 제거한 완화된 문제(Relaxed problem)에 대한 최적 목적함수 값을 뜻한다. 예를 들어 정수계획 문제에서 변수가 정수이어야 하는 조건을 없애면 선형계획 알고리즘으로 목적함수 값을 계산할 수 있다. 이렇게 계산된 목적함수 값은 원 문제의 목적함수 값보다 항상 좋을 것이다.
- 원 문제가 최소화 문제이면 목적함수 값은 경계 값보다 항상 크거나 같을 것이기 때문에 완화된문제의 목적함수 값이 하한경계(Lower bound) 실행가능한 완성스케줄 : incumbent sol.
- 원 문제가 최대화 문제이면 목적함수 값은 경계 값보다 항상 작거나 같을 것이기 때문에 완화된문제의 목적함수 값은 상한경계 (Upper bound)



분지 한계법(Branch and Bound) (5/18)

분지 한계법은 정수 계획 문제를 풀기 위한 대표적인 알고리즘이며, 암묵적 열거법이다. 분지 한계법에서는 탐색 트리(Search tree)를 이용해서 해를 찾아간다.

- 분지 한계법에서는 하한 경계 값과 현재 해를 비교해 가면서 자식 노드의 생성 여부를 결정한다. 여기서 현재 해(Incumbent solution)란 지금까지 발견된 실행 가능 해를 의미한다.
의 성능 지표 current best feasible solution. (가장 좋은)
- 만일 노드에서의 (하한 경계 값 $\geq$ 현재 해) 이면 더 이상 노드 아래를 살펴볼 필요가 없기 때문에 해당 노드를 제거(Fathoming)한다.
- 만일 노드에서의 (하한 경계 값 $<$ 현재 해) 이면 해당 노드를 제거하지 못하고 자식 노드를 생성하여 트리 탐색을 계속한다.

단일기계 일정계획 문제에서 노드, 분지, 한계는 다음과 같이 정의할 수 있다.

노드	부분 스케줄 (예: 스케줄 (???k) 에서 마지막 위치에 작업 k가 있고 나머지 위치의 작업은 결정 안됨)
분지	부분 스케줄의 결정 안된 위치에 작업 한 개 할당 (예: 마지막 위치에 작업 k가 있으면 마지막에서 두 번째 위치에 작업 i를 할당)
한계	노드의 부분 문제에 대한 하한 경계 값 계산 (예: 부분 스케줄 (??ik) 에서 첫 번째 그리고 두 번째 위치의 작업을 구하는 문제에 대한 하한 경계 값 계산)

분지 한계법(Branch and Bound) (6/18)

Branch

트리를 탐색하는 방법에는 깊이 탐색(Backtracking 또는 **Depth-first search**) 과 너비 탐색(Jumptracking 또는 **Breadth-first search**) 방법이 있다.

- **깊이 탐색**: 마지막 노드까지 깊이 탐색($V1, V2, V3$)을 하여 실행 가능 스케줄을 찾는다. 발견한 실행 가능 스케줄 (3, 2, 1)은 현재 해로 설정한다. 그리고 마지막 노드로부터 위로 올라 가면서(Backtracking) 탐색하지 않은 다른 노드 ($V4$)를 아래로 방문하고 하한 경계 값을 계산한 후 해당 노드를 제거할 지를 결정한다. 노드가 제거되면 다시 위로 올라가고 그렇지 않으면 아래로 깊이 탐색을 계속한다. 이렇듯 깊이 탐색은 우선 실행 가능 해를 먼저 만들고 노드를 방문하는 과정을 깊이 우선 순위로 진행한다.

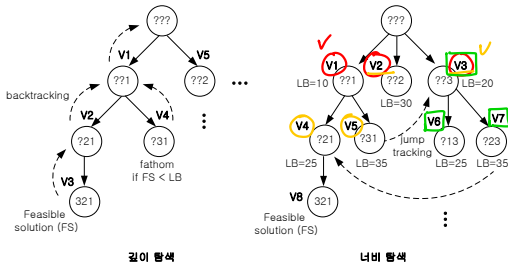
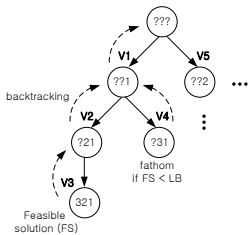


Figure 7: 트리 탐색 방법

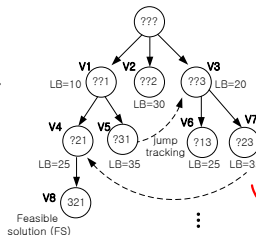
분지 한계법(Branch and Bound) (7/18)

트리를 탐색하는 방법에는 깊이 탐색(Backtracking 또는 Depth-first search) 과 너비 탐색(Jumptracking 또는 Breadth-first search) 방법이 있다.

- 너비 탐색:** 기본적으로 부모 노드에서 모든 자식 노드를 만들고 자식 노드의 하한 경계 값을 계산한다. 그리고 지금까지 분지를 하지 않은 모든 노드 중에서 가장 경계 값이 작은 노드를 선택하여 같은 과정을 반복한다. 예를 들어 루트 노드에서 자식 노드 $V1, V2, V3$ 를 만들고 각각 하한 경계 값을 계산한다. 가장 경계 값이 작은 $V1$ 을 선택하여 자식 노드 $V4, V5$ 를 생성한다. 지금까지 분지를 하지 않은 노드는 $V2, V3, V4, V5$ 이므로 가장 경계 값이 작은 $V3$ 로 이동한다(Jumptracking). $V3$ 의 자식 노드 $V6, V7$ 을 만들고 다시 분지하지 않은 노드 $V2, V4, V5, V6, V7$ 중 경계 값이 제일 작은 노드 $V4$ 를 선택한다. 그러면 실행 가능 스케줄이 발견되고 그 후 위와 같은 방법으로 너비 탐색을 진행한다.



깊이 탐색



너비 탐색

Figure 7: 트리 탐색 방법

Incumbent 찾는데 느낌.

분지 한계법(Branch and Bound) (8/18)

분지 한계법 알고리즘의 단계를 다음과 같이 정리할 수 있다.

Step 1. (Initialization) 실행 가능한 스케줄을 하나 생성 하여 현재 해로 지정한다 (초기에 해를 정하기 어려운 경우에는 big M 값을 부여해도 된다). 또한 루트 노드를 부모 노드로 지정한다. → *아무 feasible solution : incumbent solution*

Step 2. (Branching) 부모 노드로부터 자식 노드를 만들어 부분 스케줄을 생성한다.

Step 3. (Bounding) 부분 스케줄 자식 노드 의 하한 경계 값을 계산 한 후

- (하한 경계 값) $\geq$ (현재 해) 이면 해당 자식 노드를 제거한다.
- (하한 경계 값) $<$ (현재 해) 이면 분지를 통한 탐색 너비 우선 또는 깊이 우선 을 계속한다.

Step 4. 탐색 도중에 실행 가능한 스케줄을 만나고 그 해가 현재 해보다 좋으면 그 해를 새로운 현재 해로 재 설정한다. 이를 통해서 탐색의 효율성을 좀 더 높인다.

분지 한계법(Branch and Bound) (9/18)

관찰사항 4.3

분지 한계 알고리즘을 자세히 관찰해 보면 하한 경계를 계산하는 공식이 분지 한계법에서는 매우 중요하다는 것을 알 수 있을 것이다. 되도록이면 하한 경계 공식이 노드 부분 해로부터 얻을 수 있는 실행 가능 해의 최소 목적함수 값과 큰 차이(Gap)를 보이지 않는 값을 제공해야 한다. 이런 하한 경계 값을 제공하는 공식을 강력한 하한 경계 (Strongly lower bound)이라 부르며 이런 공식을 만들어야 되도록 많은 노드를 제거하여 최적 해를 발견하는 시간을 단축할 수 있다. 그러므로 강력한 하한 경계 공식을 만드는 것이 분지 한계법의 핵심이다.

(1) $\left| \sum_{i=1}^n T_i \right|$ 문제를 대상으로 다음의 두 가지 하한 경계 알고리즘을 알아본다.

- 일정계획 문제에 특화된 하한 경계
- 선형계획법을 이용한 하한 경계

분지 한계법(Branch and Bound) (10/18)

일정계획 문제에 특화된 하한 경계

- 스케줄의 마지막 위치부터 작업이 할당된 것을 부분 스케줄이라고 하고 이를 S 로 표현하자. 스케줄의 마지막 위치부터 작업을 할당하는 이유는 작업 i 의 완료시간을 쉽게 알아내서 순수지연시간 T_i 를 계산하기 위함이다. 이제 다음 용어를 정의하자.
- S' : Complement set of S **선택되지 않은 작업.**
- $P(S') = \sum_{i \in S'} p_i$ **선택되지 않은 가공시간의 합.**
- $v(S) = \sum_{i \in S} T_i$
- 예를들어, $n = 5$ 이고 부분스케줄 $S = (??312)$ 이면 $S' = \{4, 5\}$ 이고 $P(S') = p_4 + p_5$ 가 되며 $v(S) = (\sum_{i=1}^5 p_i - d_2) + (\sum_{i=1}^5 p_i - p_2 - d_1) + (\sum_{i=1}^5 p_i - p_2 - p_1 - d_3)$ 가 됨

②

①

③

분지 한계법(Branch and Bound) (11/18)

일정계획 문제에 특화된 하한 경계

- 분지는 할당 안된 마지막 스케줄 위치에 작업을 할당하여 하위 문제를 만드는 것이다. 그리고 나서 하한 경계 값을 계산해야 하는데, 이를 위해서 다음과 같은 두 가지 하한 경계 공식을 고려해 보자.

$$\textcircled{1} LB(S) = \sum_{i \in S} T_i \quad \text{and} \quad \textcircled{2} LB(S) = \sum_{i \in S} T_i + \min_{k \in S'} [\max(0, P(S') - d_k)]$$

↑
LB 보장해주기 위함. max로 하면 안돼.
S' 중 선택된 k

- 첫번째 공식은 단순히 지금까지 스케줄링 된 작업들의 순수지연시간 합을 의미한다.
- 두번째 공식에서 현재 노드에서 스케줄이 안된 작업들 중에서 마지막 위치에 있는 작업의 완료시점은 $P(S')$ 일 것이며 그 작업의 순수지연시간은 $\max(0, P(S') - d_k)$ 이 되기 때문에 현재 노드에서의 순수 지연시간의 합은 최소한 $\min_{k \in S'} [\max(0, P(S') - d_k)]$ 만큼 $\sum_{i \in S} T_i$ 보다 클 것이다.

~~$$\textcircled{1} \quad ????.2$$

$$\sum_{i=1}^k p_i - d_2$$

$$????.2$$

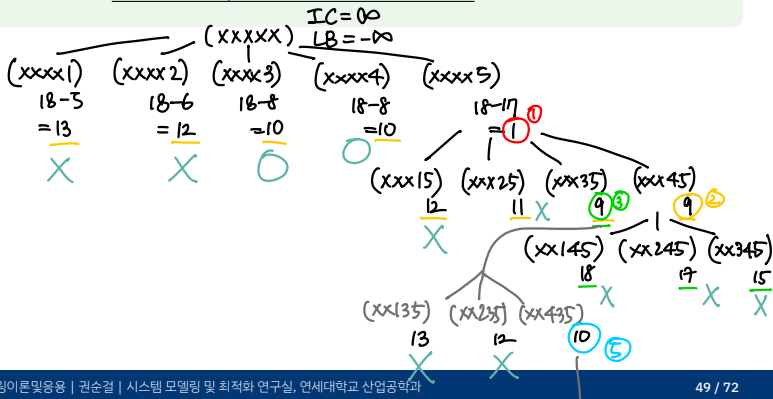
$$\sum p_i - d_2 + T_3$$~~

분지 한계법(Branch and Bound) (12/18)

예제 4.7 일정계획 문제에 특화된 하한 경계

다음의 $(1 | \sum_{i=1}^n T_i)$ 문제를 풀어보자. *네바퀴선*, $LB(s) = \sum_{ies} T_i$

작업	1	2	3	4	5
가공시간	4	3	7	2	2
납기	5	6	8	8	17



(x1435)

12

X

(x2435)

4 6

↓

(12435)

LB=11

The Maximum Lateness with Release Dates

$1|r_j|L_{\max}$ problem is important because it appears often as a subproblem in heuristic procedures for flow shop and job shop problems, but it is harder than $1|prec|L_{\max}$.

- There have been a number of effective enumerative branch-and-bound procedures developed.
- Branch and fathom a node tree
- Compute the objective of a feasible schedule (feasible node) that constructs upper bound · *Incumbent Solution*
- Compute lower bounds for a class of schedules (node)
- The node corresponding to certain schedules or classes of schedule can be fathomed if the lower bound is higher than the cost of a schedule obtained earlier

The Maximum Lateness with Release Dates

A branch-and-bound procedure for $1|r_j|L_{\max}$

- Level 0
 - There is a single node at the top of the tree.
 - No job has been put yet into any position in the sequence.
- Level 1
 - There are n branches going down to n nodes at Level 1.
 - Each node has a specific job j put into the first position in the schedule.
 - At each node, there are still $n - 1$ jobs whose position has not yet been determined.
- Level 2
 - There are $n - 1$ arcs emanating from each node at level 1 to level 2. Hence, there are therefore $(n - 1) \times (n - 2)$ nodes at level 2.
 - At each node, the jobs in the first two positions are specified.
- Level k
 - Similarly, at each node, the first k positions have been assigned to k jobs.



The Maximum Lateness with Release Dates

A branch-and-bound procedure for $1|r_j|L_{\max}$ (continued)

- Branching rule
 - It is not necessary to consider every remaining job as a candidate for the node at the next level, i.e., branching.
 - If at a node at level $k - 1$ jobs $j_1, \dots, j_{k-1}$ are scheduled as the first $k - 1$ jobs, then job j_k only has to be considered if

가장 빨리 끝나는 작업보다 도착시간이 늦은 작업을 선택
→ 비효율. cf. 한 번 스케줄

$$r_{j_k} < \min_{l \in J} (\max(t, r_l) + p_l), \quad \text{선택적 branching}$$

가장 빨리 끝나는 것
가장 빨리 시작
process

where J denotes the set of jobs not yet scheduled and t denotes the time job j_k is supposed to start. 가장 빨리 끝나는 것보다 release-time이 늦게 고려되어야 함.

- Bounding rule
 - Lower bound for a node at level $k - 1$ can be established by scheduling the remaining jobs J according to the preemptive EDD rule, i.e., relax $1|r_j|L_{\max}$ as $1|r_j, prmp|L_{\max}$. *preemption 고려하면 더 유리함 고려안하면 더 느림
 - If the preemptive solution gives a non-preemptive schedule then it is feasible.

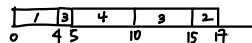
The Maximum Lateness with Release Dates

Branch-and-Bound for Minimizing Maximum Lateness

Determine the schedules that minimize the maximum lateness for the following 4 jobs.

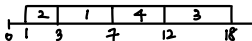
jobs	1	2	3	4
p_j	4	2	6	5
r_j	0	1	3	5
d_j	8	12	11	10

(1xxx) LB 1 3 4 2



-4, 0, 4, 5 : LB=5

(2xxx) LB 2 1 4 3

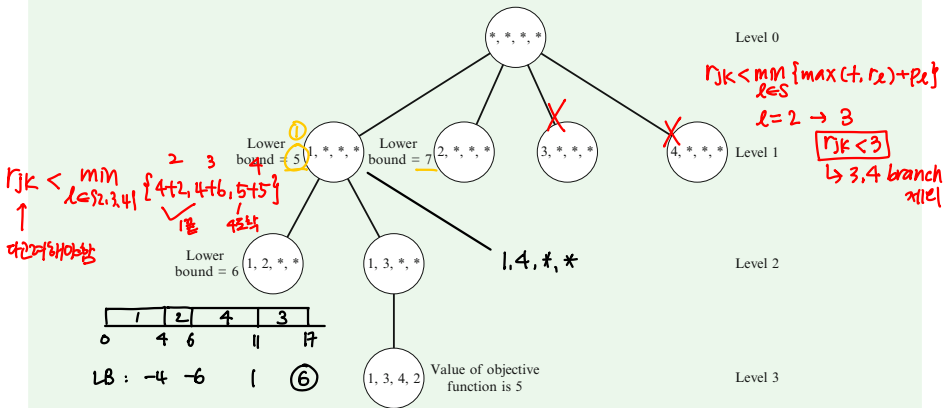


-9, -1, 2, 7 : LB=7

The Maximum Lateness with Release Dates

Branch-and-Bound for Minimizing Maximum Lateness (continued)

Search tree constructed by Branch-and-bound procedure



분지 한계법(Branch and Bound) (13/18)

선형계획법을 이용한 하한 경계

- 일정계획 문제에 특화된 하한 경계 공식을 만들기 힘든 경우에는 선형계획법을 이용한 하한 경계를 사용할 수 있다.
- 일정계획 문제를 선형계획법 문제로 공식화 하기 위해서 시간을 $0, 1, 2, \dots$ 으로 이산화 (Discretization)하여 가정하면 t 시간구간은 $[t, t - 1]$ 로 생각할 수 있다.
- 이제 Δ 를 모든 작업이 완료해야 하는 일정계획의 최대 허용시간이라 하자. 작업의 도착시간이 모두 0인 경우에는 $C_{\max}$ 는 스케줄에 상관없이 고정이며, 따라서 $\Delta = C_{\max}$ 로 정의하도록 한다.
- 선형계획법 문제의 의사결정변수, 목적함수, 제약식을 다음과 같이 정의할 수 있다.
- 의사결정변수(Decision variable)

$$x_{it} = \begin{cases} 1 & \text{작업 } i \text{가 시점 } t \text{에 가공을 시작} \\ 0 & \text{그렇지 않는 경우} \end{cases} \quad \text{time index variable}$$

- 목적함수(Objective function)

$$\text{Minimize} \quad \sum_{i=1}^n \sum_{t=0}^{\Delta - p_i + 1} (t + p_i - d_i)^+ x_{it}$$

여기서 $\sum_{t=0}^{\Delta - p_i + 1} (t + p_i - d_i)^+ x_{it}$ 는 작업 i 의 순수지연시간

분지 한계법(Branch and Bound) (14/18)

선형계획법을 이용한 하한 경계

- 제약조건(Constraints)
 - 각 작업은 $\Delta - p_i + 1$ 내에 한번만 할당되어야 한다(Assignment constraint).

$$\sum_{t=0}^{\Delta - p_i + 1} x_{it} = 1, \quad 1 \leq i \leq n$$

- 기계는 모든 시점 t 에 오직 한 개의 작업만 처리될 수 있다(Capacity constraint).

$$\sum_{i=1}^n \sum_{s=t-p_i+1}^t x_{is} \leq 1 \quad 1 \leq t \leq \Delta$$

분지 한계법(Branch and Bound) (15/18)

선형계획법을 이용한 하한 경계

- 정리된 일정계획 선형계획법 공식은 다음과 같다.

$$\text{Minimize} \quad \sum_{i=1}^n \sum_{t=0}^{\Delta-p_i+1} (t + p_i - d_i)^+ x_{it} \quad (10)$$

$$\text{subject to} \quad \sum_{t=0}^{\Delta-p_i+1} x_{it} = 1, \quad 1 \leq i \leq n \quad \text{와 진짜 클거.} \quad (11)$$

$$\sum_{i=1}^n \sum_{s=t-p_i+1}^t x_{is} \leq 1 \quad 1 \leq t \leq \Delta \quad (12)$$

$$x_{it} \in \{0, 1\} \quad \forall i = 1, \dots, n, t = 0, \dots, \Delta \quad (13)$$

- 이와 같이 $(1 \mid \sum_{i=1}^n T_i)$ 문제는 결정변수가 0 또는 1 을 갖는 이진 (Binary) 정수계획 문제로 정의된다. 이럴 때 가장 많이 사용하는 하한 경계 공식은 제약 조건 (13)을 제거하고 선형계획 문제로 간주하는 것이다. 당연히 제약조건이 하나 완화되었으므로 완화된 문제의 최적 해는 원 문제의 최적 해보다 좋을 것이다. 하지만 원 문제의 실행 가능 해는 아닌 경우가 일반적이다(즉 x_{it} 가 0 또는 1 이 아닌 실수 값 일 경우가 많다).

분지 한계법(Branch and Bound) (16/18)

선형계획법을 이용한 하한 경계

- 분지 한계 알고리즘에서 한계(Bounding)시 선형계획 문제의 최적 해를 하한 경계 값으로 사용하는 절차는 다음과 같다.

Step 1. (Initialization) 루트 노드에서 제약 조건 (13)를 제거한 문제를 선형계획 알고리즘(Simplex 알고리즘)을 이용해서 푼다. 결정변수 x_{it} 가 모두 0 또는 1 이면 종료한다(최적해 발견).

Step 2. (Branching) 실수 값을 갖는 변수를 하나 선택하여($x_{it} = k$ 라고 하자), $x_{it} \geq \lceil k \rceil$ 제약조건과 $x_{it} \leq \lfloor k \rfloor$ 제약조건을 만들어 자식 노드를 생성한다.

Step 3. (Bounding) 각 자식 노드마다 추가된 제약조건을 고려하여 다시 선형계획 알고리즘을 이용해서 하한 경계 값을 구한다. 만일 (하한 경계 값) $\geq$ (현재 해) 이면 해당 자식 노드를 제거한다. (하한 경계 값) $<$ (현재 해) 이면 분지를 통한 탐색 너비 우선 또는 깊이 우선 을 계속한다.

Step 4. 탐색 도중에 실행 가능한 스케줄(결정변수가 모두 0 또는 1) 을 발견하고 그 해가 현재 해보다 좋으면 그 해를 새로운 현재 해로 재 설정한다.

분지 한계법(Branch and Bound) (17/18)

예제 4.8 선형계획법을 이용한 하한 경계

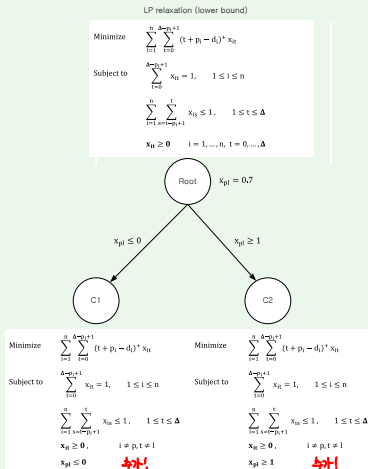


Figure 8: LP를 이용한 분지 한계 예제

분지 한계법(Branch and Bound) (18/18)

선형계획법을 이용한 하한 경계

관찰사항 4.4

- 앞서 정의한 최적화 모형에서 성능지표인 총 작업 순수지연시간 $\sum_{i=1}^n T_i$ 을 다른 종류의 Total이나 Average형태의 지표로 수정해도 선형계획법을 이용한 분지 한계법을 이용하면 최적 스케줄을 발견할 수 있다.
- 선형계획법을 이용한 분지 한계법은 결정변수의 선정과 이에 따른 최적화 공식에 따라서 큰 성능 차이를 보인다. 앞서 소개한 최적화 공식은 선형계획법으로 풀면 매우 강한 하한 경계를 제공하기 때문에 널리 사용된다. 그러나 제약조건이 $n + \Delta$ 개 있고 변수가 $n\Delta$ 만큼 존재하기 때문에 n 이 10개 이상만 되더라도 최적 해를 찾는 데 너무 많은 시간이 소요된다는 단점이 있다. 따라서 최적 해를 찾는 데 가장 좋은 방법은 문제에 특화된 하한 경계 공식을 만드는 것이다. 하지만 이는 매우 어렵고 고된 일이다.

메타 휴리스틱 알고리즘 (1/2) $\leftrightarrow$ 휴리스틱

↓
problem-specific

메타 휴리스틱 알고리즘은 운이 좋으면 전역 최적 해(Global optimum)를 발견할 수도 있지만, 일반적으로 국지 최적 해(Local optimum) 비교적 빠른 시간 내에 찾는 것을 목적으로 한다.

- 목적함수가 Total 또는 Average를 가진 일정계획 문제는 단일기계일자라도 NP-hard에 속하므로 동적 계획법과 분지 한계법등의 암묵적 열거법을 사용하더라도 문제 크기가 커지면 전역 최적 해를 찾는 데 너무나 오랜 시간이 걸린다.
- 메타 휴리스틱 알고리즘은 이웃 해 탐색 방법에 기반을 둔다.

Step 1. 탐색 단계를 $t = 1$ 로 놓고 초기 해 S_t 을 만든다. 가장 좋은 해를 S_B 라 하고 $S_B \leftarrow S_t$ 로 놓는다.

Step 2. 이웃 해 생성 방법을 이용하여 현재 해 S_t 의 이웃 해 S_C 의 집합을 생성한다.

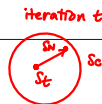
Step 3. 생성된 이웃 해 집합 내에서 하나를 선택해 S_N 으로 놓는다. 만일 S_N 이 S_B 보다 좋으면 $S_B \leftarrow S_N$ 으로 한다. ↗ 해의 개선

Step 4. 종료조건이 만족할 때까지 $t = t + 1$, $S_t \leftarrow S_N$ 으로 놓고 Step 2를 반복한다.

- 여기서 가장 좋은 해 S_B 는 t 번째 탐색까지 발견한 해중에서 가장 좋은 해가 된다. 그렇지만 이 알고리즘에서 현재 해 S_t 의 이웃 해 집합이란 해 공간 전체에서 S_t 근처에 있는 아주 작은 부분 집합이므로 결국 가장 좋은 해인 S_B 는 t 번째 탐색까지 살펴본 이웃 해 집합들로 구성된 일부 해 공간 내에서만 최적이다.

메타 휴리스틱 알고리즘 (2/2)

메타 휴리스틱을 위한 필요한 이웃해 탐색 절차에 사용된 기호 정리



기호	iteration 의미
S_t	탐색 단계 t 에서의 현재 해 (즉 $t-1$ 단계에서의 S_N).
S_C	탐색 단계 t 에서 S_t 의 이웃 해.
S_N	탐색 단계 t 에서 S_t 의 이웃 해 중 선택된 해.
S_B	탐색 단계 0부터 t 까지 선택된 스케줄 중에서 가장 좋은 해.

- 타부 탐색(Tabu search)과 시뮬레이티드 어닐링(Simulated annealing)은 Step 3의 이웃 해 선택을 개선한 대표적인 메타 휴리스틱 알고리즘이다. 이 두 방법은 국지 최적 해에 만족하지 않고 전역 최적 해를 찾아가는 기발한 아이디어를 체계적으로 정리한 것이며 구체적으로 설명하면 두 알고리즘은 이웃 해를 선택할 때 성능지표 값이 나쁜 해도 가끔씩 선택 될 수 있도록 한다.
- 일반 이웃 해 탐색과 같이 지속적으로 해가 개선되는 방향으로 나아가는 것을 **Intensification**이라 하고 해가 안 좋은 방향으로도 이동하는 것을 **Diversification**이라 한다. Intensification 은 특정 지역 내에서만 집중적으로 탐색을 하게 만들고 **exploration** Diversification 은 여러 지역을 돌아다니게 만드는 효과가 있다.
- 타부 탐색과 시뮬레이티드 어닐링은 Intensification 과 Diversification 둘 간에 조화를 맞춘다는 아이디어는 같으나 이런 조화를 어떻게 구현할 것이냐는 다르다.

어떻게 구현?

타부 탐색을 이용한 일정계획 (1/4)

★ 일정계획 문제에서 타부 탐색의 기본 아이디어는 최근에 선택한 스케줄 S_N 을 기억하고 있다가 나중에 이웃 스케줄을 선택할 때(Step 3) 이 스케줄로 돌아가지 않도록 하는 것

- 최근에 선택된 스케줄을 기억하고 있는 목록을 **타부 목록(Tabu list)**이라고 부르며 제한된 크기의 **FIFO(First In, First Out)**구조의 리스트에 저장한다. 타부 목록에 한번 저장된 내용은 일정 기간 동안 Step 3 에서 이웃 스케줄을 선택할 때 제외된다. 이렇게 함으로써 이미 방문한 스케줄로 다시 돌아오지 못하게 하여 국지 최적 해로 회귀하는 것을 방지한다.
- 만일 탐색 단계 t 의 Step 2에서 생성한 이웃 스케줄 중에서 오직 하나만이 현재 스케줄 S_t 보다 좋으나 최근에 이미 방문한 스케줄이라 타부 목록에 있으면 Step 3 에서 선택하는 스케줄 S_N 은 S_t 보다 안 좋은 것일 수 있다. 나쁜 스케줄을 선택했다는 것은 탐색 공간에서 새로운 영역으로 이동하여 탐색을 새롭게 시작한다는 의미로 받아들이면 된다.
- 타부 목록의 길이는 제한되어 있고 FIFO 구조이기 때문에 저장된 내용은 언젠가는 목록에서 빠져나오게 되어 다시 선택 대상이 될 수 있다. 여기서 생각해 볼 수 있는 점은 타부 목록이 길어지면 Diversification이 증가되고 반대로 짧아지면 Intensification이 증가한다는 것이다. 타부 목록의 길이는 풀고자 하는 문제의 해 공간이 얼마나 크냐에 따라서 달라지나 일반적으로 7 에서 20 사이로 설정 한다.

다이버시피케이션 vs 익스플로러 vs 익스플로이트 정도 조절

- size ↑ : explore ↑
- size ↓ : exploit ↑

타부 탐색을 이용한 일정계획 (2/4)

일정계획 문제에서 타부 탐색의 기본 아이디어는 최근에 선택한 스케줄 S_N 을 기억하고 있다가 나중에 이웃 스케줄을 선택할 때(Step 3) 이 스케줄로 돌아가지 않도록 하는 것

- 타부 목록에 스케줄을 저장하는 방법을 예를 들어 설명하면 다음과 같다. 탐색 단계 t 에서 현재 스케줄 $S_t = (2, 1, 4, 3)$ 이 있고 Step 3에서 선택된 이웃 스케줄이 $S_N = (2, 4, 1, 3)$ 이라면
 - (1) 현재 스케줄 전체인 $S_t = (2, 1, 4, 3)$ 을 타부 목록에 넣을 수 있다. 이러면 S_t 가 타부 목록에 있는 한 스케줄 S_t 로 돌아가는 것을 허락하지 않겠다는 것이다.
 - (2) 보편적으로 사용하는 방법은 교환된 작업만 저장 하는 것이다. 위의 예제에서 작업 1과 4가 교환됐으므로 (1, 4)가 저장 내용이 된다. 즉 당분간 작업 1이 작업 4에 선행하는 스케줄은 허락하지 않겠다는 것이다.
- (1)번 방법 보다 (2)번 방법이 좀 더 강한 금지 효과를 지진다. 예를 들어 (1)번 같은 경우는 스케줄 (2, 1, 4, 3) 만을 금지하지만 (2)번의 경우는 (1, 4)가 들어간 모든 스케줄 (예를 들어 (2,1,4,3), (1,4,2,3), (3,2,1,4) 등)을 금지한다.
- (1)번 과 같이 해 전체를 저장하는 방식을 Short-term memory 라고 부르고 (2)번과 같이 해의 일부분만을 저장하는 것을 Intermediate-term memory 라고 한다. Intensification과 Diversification 측면에서 보면 Short-term memory 은 Intensification 을 선호하고 Intermediate-term memory 은 Diversification 을 선호하는 경향이 있다. → (2)가 diversification에 더 특화

타부 탐색을 이용한 일정계획 (3/4)

일정계획 문제를 위한 타부 탐색 알고리즘의 절차

- 기본적으로 타부 탐색은 앞에서 살펴본 이웃 해 탐색 방법과 Step 3 를 제외하고는 같다. 그리고 타부 탐색에서의 Step 3 는 다음과 같이 정리할 수 있다.

Step 3. 타부 탐색에서의 스케줄 선택

Step 3.1. 현재 스케줄 S_t 를 기준으로 생성된 이웃 해들을 성능지표에 의거해 (오름차순) 정렬하고 첫 번째 후보 스케줄 S_C 를 선발한다. 만일 $\text{Move } S_t \rightarrow S_C$ 가 타부 목록에 있으면 다음 스케줄을 후보 스케줄 S_C 로 놓고 타부 검증을 반복한다. $\text{Move } S_t \rightarrow S_C$ 가 타부 목록에 없는 스케줄 S_C 을 선택하고 이를 선택 해 S_N 으로 놓는다. 이 과정에서 현재 해보다 안 좋은 해가 선택될 수 있다.

Step 3.2. $\text{Move } S_t \rightarrow S_C$ 을 타부 목록에 기록한다. 목록이 꽉 차 있으면 가장 오래된 Move 를 타부 목록에서 제거한다.

Step 3.3. 만일 S_N 이 S_B 보다 좋으면 $S_B \leftarrow S_N$ 으로 한다.

- 타부 탐색에서는 하나의 스케줄 로부터 다른 스케줄이 만들어지는 과정 또는 위치교환 연산을 Move 라고 표현한다. $S1 = (1, 2, 3, 4)$ 이고 $S2 = (2, 1, 3, 4)$ 라면 $\text{Move } S1 \rightarrow S2$ 는 (2, 1) 이 된다.

타부 탐색을 이용한 일정계획 (4/4)

예제 4.9 타부 탐색을 이용한 일정계획

다음의 $(1 | \sum_{i=1}^n w_i T_i)$ 문제를 풀어보자.

작업	1	2	3	4
가공시간	10	10	14	4
납기	4	2	1	12
가중치	14	12	1	12

iteration	S_t (성능지표)	이웃해상합 (성능지표)	선택된 이웃해 S_N	(size 2) Tabu List	가장좋은 해 S_B
1	(2143) 500	(1243) 480 ② (2413) 436 ① (2134) 652 ③	(2413)	{(1,4)}	500 → 436 (2413)
2	(2413) 436	(4213) 460 ① (2143) 500 ② (2431) 608 ③	(4213)	{(1,4), (2,4)}	436 (2413)
3	(4213) 460	(2413) (4123) 440 (4231) 632	(4123)	{(2,1), (2,4)}	436 (2413)
↓ (...)					

시뮬레이티드 어닐링을 이용한 일정계획 (1/5)

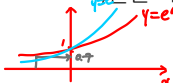
시뮬레이티드 어닐링도 타부 탐색처럼 이웃 해를 선택할 때 성능지표 값이 안 좋은 해도 선택될 수 있도록 하는것이 기본 아이디어

- 금속의 원자에 가열을 하면 본래 위치 초기 스케줄에 해당함 보다 에너지(성능지표 값)가 높아져 매우 불안정하게 되며 냉각을 천천히 시키면 에너지가 줄어들면서 원래 위치보다 더 단단한 금속 구조체(성능지표 값이 매우 좋은 스케줄)가 만들어지는 물리적 담금질 (Annealing) 현상을 모방 (Simulation)한 것이다. 이런 아이디어 때문에 시뮬레이티드 어닐링(Simulated annealing)이라 명명한 것이다.
- 시뮬레이티드 어닐링은 안 좋은 해를 선택하도록 하는 관용의 폭을 알고리즘의 실행 초기에는 넓히고(Diversification 선호) 실행 후반기에는 좁히는(Intensification 선호) 디자인 방식을 택하여 전역 최적 해를 찾아가도록 한다.
- 실행 초기에는 선택한 스케줄 S_N 의 성능지표 값이 위 아래로 출렁이다가 나중에는 값이 일정하게 아래로 낮아 진다(수렴하게 된다).

시뮬레이티드 어닐링을 이용한 일정계획 (2/5)

시뮬레이티드 어닐링은 이런 물리적 현상을 이용하여 현재 스케줄 S_t 로부터 스케줄 S_N 을 확률적으로 선택한다(가열에 해당함).

- 구체적으로 설명하면 S_t 의 이웃 스케줄 S_C 가 S_t 보다 좋은 스케줄이면 S_N 이 된다 (즉 $S_N \leftarrow S_C$). 그리고 S_C 가 S_t 보다 나빠도 채택 확률 (Acceptance probability)에 의거하여 선택될 수 있도록 한다. 스케줄 S 의 성능지표 값을 $f(S)$ 라고 정의하면 채택 확률은 다음과 같다.



$$P_t(S_t, S_C) = \exp \left(\frac{f(S_t) - f(S_C)}{\beta_t} \right), \quad \text{where } f(S_t) - f(S_C) \leq 0$$

안 좋은수록 허용확률값이 ① 0~1사이 ② 작아져야함.

(①반면은 a가 작은 값을주고, diver. 방지)
(②반면은 a↑, intensification↑)

- 이식을 보면 S_C 가 아주 나쁜 스케줄이면(즉 $f(S_t) \ll f(S_C)$ 이라는 의미) 지수함수의 성질에 의해서 채택확률은 매우 작을 것이고 반대로 S_C 가 S_t 에 견줄만한 스케줄이면(즉 $f(S_t) \approx f(S_C)$ 이라는 의미) 채택확률은 커질 것이다. 만일 S_C 가 확률적으로 채택되지 않으면 다른 이웃 스케줄을 대상으로 확률적 채택과정을 반복한다.
- 이 식에서 눈 여겨 봐야 할 기호가 있는데 그것이 냉각 모수 (Cooling parameter) β_t 이다. 이 모수는 온도를 의미하므로 탐색이 진행되면서 값이 작아질 것이다 ($\beta_1 \geq \beta_2 \geq \dots \geq 0$). 따라서 β_t 가 크면(즉 초기 가열에 해당함) S_C 가 채택될 확률이 높아지고 β_t 가 작으면 채택 가능성은 낮아진다(후기 냉각에 해당함). 따라서 처음에는 스케줄의 대체가 활발하게 진행되다가 온도가 식으면서 점점 좋은 스케줄을 찾아가면서 안정화 될 것이다.

시뮬레이티드 어닐링을 이용한 일정계획 (3/5)

일정계획 문제를 위한 시뮬레이티드 어닐링 알고리즘 절차

- 기본적으로 시뮬레이티드 어닐링 알고리즘은 앞에서 살펴본 이웃 해 탐색 방법과 Step 3를 제외하고는 같다. 그리고 시뮬레이티드 어닐링 알고리즘에서의 Step 3는 다음과 같이 정리할 수 있다.

Step 3. 시뮬레이티드 어닐링에서의 스케줄 선택

Step 3.1. 현재 스케줄 S_t 를 기준으로 생성된 이웃 해들을 성능지표에 의거해 (오름차순) 정렬하고 첫 번째 후보 스케줄 S_C 를 선발한다. $f(S_t) > f(S_C)$ 이면 $S_N \leftarrow S_C$ 로 놓는다. 그러나 $f(S_t) \leq f(S_C)$ 이면 채택 확률 $P_t(S_t, S_C)$ 에 의거해 S_C 를 선택할지 결정한다(이 과정에서 현재 해보다 안 좋은 해가 선택될 수 있다). 선택되면 $S_N \leftarrow S_C$ 으로 놓고 선택되지 않으면 다음 후보 스케줄을 대상으로 확률적 채택 과정을 반복한다.

Step 3.2. 만일 S_N 이 S_B 보다 좋으면 $S_B \leftarrow S_N$ 으로 한다.

시뮬레이티드 어닐링을 이용한 일정계획 (4/5)

$$\beta_t \leftarrow \beta(t) \text{ 함수.}$$

냉각 모수의 함수 형태

냉각 모수는 시간 t 에 따라 값이 감소하는 함수로 정의하고 이를 냉각 스케줄 (Cooling schedule)이라 부르며 일반적으로 많이 사용하는 세 가지 함수는 다음과 같다.

- 로그 함수: $\beta_t = \frac{\beta_0}{\ln(1+t)}$
여기서 β_0 는 초기 온도로 문제마다 다르게 정해야 한다. 이 식은 로그 함수의 성질에 의해서 냉각이 서서히 이루어지도록 만든다. 따라서 Diversification 을 오래 유지하도록 만드는 경향이 있다.
- 기하 함수: $\beta_t = \alpha^t \beta_0$
여기서 α 는 냉각 속도를 조절하는 또 다른 모수로 $0 < \alpha \leq 1$ 를 만족해야 한다. 보통 α 는 0.8과 0.99 사이 값을 갖도록 한다. 모수 α 가 이 사이 값을 가지면 냉각이 일직선으로 이루어진다.
- 지수 함수: $\beta_t = \beta_0 \exp\left(-\alpha t^{\frac{1}{n}}\right)$ where $0 < \alpha < 1$ and n is the dimensionality of solution space
여기서 n 은 해 공간의 크기를 의미하며 단일기계 일정계획에서는 작업의 수이다. 이 함수는 기하 함수보다 좀 더 빨리 냉각이 되도록 만든다.

시뮬레이티드 어닐링을 이용한 일정계획 (5/5)

냉각 모수의 함수 형태 (continued)

로그함수, 기하함수, 지수함수에 따른 냉각 속도 변화 양상

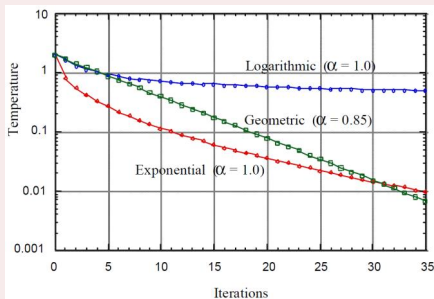


Figure 9: 냉각 모수에 따른 냉각 속도 변화

유전자 알고리즘을 이용한 일정계획 (1/6)

유전자 알고리즘은 인간의 진화과정을 모방한 알고리즘이다.

- 인간이 유전자의 서열인 염색체로 구성되어 있듯이 유전자 알고리즘에서는 하나의 해를 염색체 (Chromosome) 또는 개별 해(Individual)라고 한다. 각 염색체는 스트링 (String) 구조를 가지며 스트링내의 각 단위를 유전자 (Gene)라고 부른다. 예를 들어 단일기계 일정계획 문제에서 염색체는 하나의 스케줄 스트링 (1, 2, 3, 4)이 되고 스케줄에 위치한 각 작업이 유전자에 해당된다.
- 그리고 염색체는 적합도 (Fitness)라는 염색체의 우수성을 보이는 값이 따라 붙는다. 일정계획 문제에서 스케줄의 적합도는 당연히 스케줄의 성능지표 값이다. 혹은 N 개의 스케줄이 있을 때 성능지표 값을 오름차순으로 정렬한 후 가장 좋은 스케줄 성능지표 값이 제일 작은 스케줄에 적합도 값 N 을 부여하고 그 다음 좋은 스케줄에 $N-1$ 적합도를 주는 식으로 진행하는 서열 방식도 있다.
- 서열로 적합도를 부여하는 방식은 성능지표로 적합도를 정하는 방식보다 염색체의 우수성을 확실하게 분류할 수 있다는 장점이 있으나 어느 정도의 우수성을 가졌는지는 표현할 수 없다. 적합도에 염색체의 서열이 아니라 우수성의 정도를 표현하려면 혹은 그러한 정보가 문제를 푸는데 중요하다면 성능지표 값을 적합도로 사용해야 한다.

유전자 알고리즘을 이용한 일정계획 (2/6)

아주 초기 탐색
pop to pop

유전자 알고리즘은 인간의 진화과정을 모방한 알고리즘이다.

- 인간 세상에서 한 세대를 이루는 무리가 있듯이 유전자 알고리즘에서도 한 세대 (Population)는 다수의 염색체로 구성되어 있다. 그리고 인간이 세대를 거듭할수록 적자생존의 원칙에 따라서 좀 더 나은 존재로 발전하듯이 이 알고리즘에서도 부모 세대에서 자식 세대로 넘어가면서 적합도가 좀 더 좋은 염색체들로 세대가 구성된다. 한 세대에서 다음 세대로 넘어가는 것을 탐색 단계 t 로 볼 때 유전자 알고리즘에서는 많은 세대 변화를 거치면 최종 세대에는 매우 우수한 염색체만 남게된다.

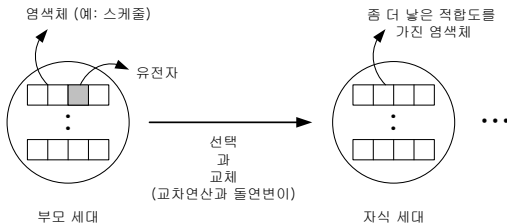


Figure 10: 유전자 알고리즘의 개념

유전자 알고리즘을 이용한 일정계획 (3/6)

유전자 알고리즘에서 세대의 진화는 두 가지 방법에 의해서 이루어진다.

- (1) **선택(Selection)**: 부모 세대에 있는 좋은 염색체는 선택되어 다음 세대에 그대로 옮겨간다. 부모 세대가 N 개의 염색체로 구성되어 있으면 $N/2$ 이상이 선택되도록 디자인한다. 좋은 염색체를 선택하는 방법으로는 적합도가 좋은 것 만 바로 선택하는 확정적인 방법과 다음과 같이 적합도를 가지고 확률적으로 선택하는 방법이 있다.
- (2) **교체(Replacement)**: 선택되지 못한 나머지 부모 염색체는 다음 세대까지 가지 못하고 사라진다. 대신에 새로운 염색체가 유전자 연산을 통해서 다음 세대에 등장하게 된다. 유전자 알고리즘의 핵심은 두 종류의 유전자 연산에 있다. 하나는 교차(Crossover) 연산이라는 것으로 부모 염색체 쌍의 유전자를 변환하여 새로운 자식 염색체 쌍을 만드는 것이고, 다른 하나는 돌연변이(Mutation) 이다.

유전자 알고리즘을 이용한 일정계획 (4/6)

유전자 연산

- 교차 연산: 여러 종류의 교차 연산이 제안되었지만 본 강의에서는 단일기계 일정계획에 적합한 교차마스크 (Crossover mask) 방법을 소개한다. 교차마스크는 일종의 템플릿으로 스케줄에서 순서를 바꾸지 않을 위치를 정한 것이다. 예를 들어 마스크가 (1000) 이면 스케줄에서 첫 번째 작업은 그대로 두고 나머지 위치에 있는 작업들을 교환한다는 뜻이다. 교차마스크를 어떻게 구성할지에 따라서 Single point crossover, Two point crossover, 그리고 Uniform crossover 가 있다.

	부모 염색체	교차 마스크	자식 염색체
Single-point crossover	(1, 2, 3, 4, 5)	(11000)	(1, 2, 4, 5, 3)
	(2, 5, 3, 1, 4)		(5, 2, 3, 1, 4)
Two-point crossover	(1, 2, 3, 4, 5)	(00110)	(5, 1, 3, 4, 2)
	(2, 5, 3, 1, 4)		(2, 5, 1, 3, 4)
Uniform crossover	(1, 2, 3, 4, 5)	(10110)	(1, 5, 3, 4, 2)
	(2, 5, 3, 1, 4)		(3, 5, 1, 2, 4)

유전자 알고리즘을 이용한 일정계획 (5/6)

유전자 연산

- 돌연변이: 교차 연산은 부모 염색체 쌍의 유전자를 변환하여 새로운 자식 염색체 두 개를 생성하는 것에 비해 돌연변이는 자식 세대에 있는 스케줄에 돌연변이 연산을 실행한다. 구체적으로 자식 세대에 있는 스케줄을 선택하고 스케줄상의 위치 두 개를 임의로 선택한 후 해당 위치에 있는 작업을 교환하는 방법이다. 이는 이웃 해 생성 방법의 Single pairwise interchange 와 동일하다.

자식 염색체	돌연변이 위치 선정	돌연변이 자식 염색체
(2, 5, 3, 1, 4)	2, 5	(2, 4, 3, 1, 5)

- 유전자 알고리즘은 타부 탐색과 시뮬레이티드 어닐링과 달리 범용성이 크기 때문에 알고리즘을 실행하기 전에 결정해야 할 모수가 세 개 존재한다.

세대크기(N)	모든 세대는 구성 염색체 수가 N 으로 동일 하다.
$r \uparrow$ diversification 교차비율(r)	부모 세대에서 $rN/2$ 쌍이 선택하여 교차 연산 을 수행한다. 그리고 $(1-r)N$ 수 만큼의 염색체는 선택 되어 자식 세대로 옮겨간다.
$u \uparrow$ diversification 돌연변이 비율(u)	자식 세대의 염색체에 돌연변이를 실행할 비율이다.

유전자 알고리즘을 이용한 일정계획 (6/6)

유전자 알고리즘의 절차

Step 1. 탐색 단계를 $t = 1$ 로 놓고 초기 스케줄을 N 개 만들어 부모 세대 P_t 를 구성한다.

Step 2. (Selection) 부모 세대 P_t 에서 적합도를 기준으로 스케줄을 오름차순으로 정렬한 후 $(1-r)N$ 수 만큼의 스케줄을 선택하여 자식 세대 P_{t+1} 에 넣는다.

Step 3. (Replacement: Crossover) 부모 세대에서 $rN/2$ 쌍의 스케줄을 임의 (Random)로 선택하여 교차 연산을 실시하고 새롭게 생성된 $rN/2$ 쌍의 자식 스케줄을 P_{t+1} 에 넣는다.

Step 4. (Replacement: Mutation) 자식 세대 P_{t+1} 에 속한 스케줄 중에서 임의로 uN 개를 선택하고 돌연변이 연산을 수행한다.

Step 5. 종료조건이 만족할 때까지 $t = t + 1$ 로 놓고 Step 2를 반복한다.

관찰사항 4.5

타부 탐색 과 시뮬레이티드 어닐링은 탐색 단계를 거치면서 해가 하나씩 선택되는 이웃 해 생성 방법을 따르지만 유전자 알고리즘은 각 단계마다 해 집단 세대를 생성한다는 점에서 앞서 공부한 두 알고리즘과 다르다. 그러므로 유전자 알고리즘은 타부 탐색이나 시뮬레이티드 어닐링보다 보다 폭넓은 Diversification 을 허용한다. 그러나 유전자 알고리즘은 일반적으로 두 알고리즘보다 해를 개선해 나가는 속도가 느리다는 단점이 있다.

Any Questions?

권순걸(sg.kwon@yonsei.ac.kr)

시스템 모델링 및 최적화 연구실, 연세대학교 산업공학과

SYstems **M**odeling and **P**rogramming Lab
Department of Industrial Engineering, Yonsei University - **SYMPLY**
(<https://symply.yonsei.ac.kr/>)

